Safe Constrained Reinforcement Learning for Precision Grasping on a UR5e Manipulator: A Simulation Study

A thesis submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science

in

Mechatronics Engineering

by

Md. Abdur Rabbi

Supervised by

Priyo Nath Roy

Department of Mechatronics Engineering

Khulna University of Engineering & Technology

Khulna-9203, Bangladesh

Date of Defense: August 2026

CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

CMDP	Constrained Markov Decision Process
cPPO	Constrained Proximal Policy Optimisation (PPO-Lagrangian)
DOF	Degree of Freedom
GPU	Graphics Processing Unit
IBVS	Image-Based Visual Servoing
MDP	Markov Decision Process
PPO	Proximal Policy Optimisation
RL	Reinforcement Learning
SAC	Soft Actor–Critic
TCP	Tool Centre Point
TD3	Twin Delayed Deep Deterministic Policy Gradient
USD	Universal Scene Description

Symbols

c	Per-step safety cost
d	Episodic cost budget
J	End-effector Jacobian
λ	Lagrange multiplier on the safety constraint
η	Dual learning rate
γ	Discount factor
w	Yoshikawa manipulability measure, $\sqrt{\det(JJ^\top)}$
π	Policy
rad	Radian

CHAPTER 1

INTRODUCTION

A manipulator is, in its simplest description, a chain of rigid links connected by powered joints, ending in a tool that does the actual work of picking something up, placing it, or holding it against a process. For most of industrial robotics' history this chain worked alone, fenced off from people, repeating a single memorised path thousands of times a shift. The collaborative arm changes that arrangement: it is built to share a workspace with a person, which means every motion it makes must be treated as something that could come near a hand, not just near a workpiece. This thesis is built around one specific arm of that kind, and the platform is not incidental to the argument. Everything the simulation trains against, from the joint speeds it is allowed to command to the distances it is allowed to close, is set to match what this particular machine can actually do.

Figure 1.1. A UR5e collaborative arm sharing a workspace with a person. Photograph reproduced from Xia et al. [6], Fig. 1, under that paper's CC BY-NC-ND 4.0 licence. Placeholder pending an original photograph of the platform used in this project.

The platform is a Universal Robots UR5e, a six-joint (6-DOF) collaborative arm [9]. It carries a payload of 5 kg to a reach of 850 mm, repeats a taught position to within 0.03 mm, and weighs 20.6 kg unmounted. Every one of its six joints is rated to the same maximum speed, 180 deg/s, which is exactly pi radians per second. The simulated arm in this thesis is capped at 3.14 rad/s for the same reason a real one is: past that speed the motion stops being something a person nearby could react to in time. That number is not a simulation

convenience chosen for training stability; it is the arm's own datasheet limit, carried into the simulator so that a policy trained there is never asked to move faster than the physical machine could follow it. The UR5e also ships with seventeen configurable safety functions and is certified to the relevant collaborative-robot safety standards, ISO 13849-1 (Category 3, Performance Level d) and ISO 10218-1 [9]. None of those seventeen functions is exercised by the simulation. What this thesis borrows from the platform is its physical envelope, its speed and reach limits, not its built-in safety controller, and Chapter 3 is explicit about which of the two is under test.

Machines with this specification already do real work outside the laboratory. UR-class arms in the UR5e's size class are common on production floors for exactly the tasks a 5 kg payload and an 850 mm reach suit: pick-and-place, machine tending, small-parts assembly and packaging, and inspection stations where a part has to be picked up and held in front of a camera. The same collaborative certification that lets one work next to a person on a line is also why it turns up so often in research: it is affordable enough for a university laboratory, safety-rated enough to run near people without a fenced cell, and common enough on real production floors that a result obtained on one arm should say something useful about the rest of that population. Two of the papers already cited in this chapter used a UR5-class arm for exactly that reason, one for grasping [8] and one for collision avoidance around a moving person [6]. The platform used here is drawn from that same working population.

The choice of platform is not arbitrary either. The preceding project in this laboratory used a different, custom five-degree-of-freedom arm for classical visual servoing [10]; this thesis moves to a UR-class platform because the safety question it asks, how a learned policy behaves near a joint limit or a kinematic singularity, is best posed on hardware whose limits and safety record are already public and well documented. Isaac Lab, the simulation framework used throughout (Chapter 3), also ships the UR5e as one of its reference manipulators, so building the environment on this arm meant working from a supported example rather than adapting a custom robot from scratch. The joint limits and speed ceiling used throughout this thesis therefore belong to a machine that exists, not one assumed for convenience.

1.1 Background

Industrial manipulators increasingly work in spaces shared with people rather than behind a fence, and the UR5e described above is a working example of that shift, not a special case of it. A controller that follows a fixed, hand-derived trajectory can be inspected and bounded before it ever moves: an engineer can trace every configuration it will visit and check each one against a joint limit or a known obstacle. A policy obtained by training a reinforcement learning agent cannot be inspected the same way. Its behaviour is known only through the statistics of the trajectories it produces during and after training, not through a proof that

covers every configuration it might reach. Safe operation of a UR-class collaborative arm around people is already treated as a live industrial requirement, not a hypothetical one [6]. Reinforcement learning is attractive for manipulation precisely because it can learn grasping and placement behaviour directly from experience, without a hand-built controller for every object and pose the arm might encounter. What that learned behaviour does in the configurations it visits rarely, however, is not a detail to be checked after the fact. For a policy sharing a workspace with a person, it is the condition deployment depends on.

The standard formulation of reinforcement learning gives a designer only one channel for stating what the agent should avoid: the reward function. Take the UR5e's own joint-speed ceiling as a concrete case. A designer who wants the trained policy to respect it has, in the standard formulation, exactly one lever: add a penalty term for exceeding it, sized by a weight that trades a unit of excess speed against a unit of task reward. That weight has to be fixed before training starts, before the designer has any real sense of how large a violation the untrained policy will produce or how much task performance the penalty will cost once it starts to bind. Set the weight too small and the constraint is decorative. Set it too large and the agent may refuse to move at all. An alternative, pursued in this thesis, is to keep the requirement outside the reward entirely and state it instead as a constraint with an explicit budget: a quantity that can be set from a measurement of the system, such as the arm's own rated limits, and checked afterwards against what the trained policy actually used [11]. The distinction matters because of what each side has to answer for. A reward weight is a guess, tuned by trial and error against whatever the agent happens to do. A constraint budget is a number the designer can defend before training ever starts, because it comes from a measurement rather than a hunch. Chapter 2 reviews this distinction, and the literature built on either side of it, in full.

1.2 Problem Description

This thesis puts that alternative to a direct test. It trains a UR5e in simulation to pick up and lift a cube, comparing a standard proximal policy optimisation (PPO) agent against a constrained variant of the same algorithm (PPO-Lagrangian) under a shared floor on manipulability, a measure of how far a configuration sits from a kinematic singularity. Such a comparison, as usually run in the literature, has two problems.

- **The cost-critic confound.** Adding a constraint to an RL agent does not add only a constraint. A Lagrangian method also adds a cost critic, a second value network trained alongside the policy to estimate the constraint violation the current policy would incur. That network is present whether or not the constraint is actively binding, and its presence can influence training through machinery the two agents no longer share. A comparison that trains only the unconstrained baseline and the fully constrained

agent cannot separate what the constraint achieved from what the extra network alone changed.

- **Seed variance.** A single training run under one random seed says little about what a method will do on the next run [4]. Published comparisons of constrained against unconstrained manipulation policies typically report one run, or a mean over few, without a measure of how much the outcome would vary if the same method were retrained.

Section 2.11 sets out this gap in full against the closest published comparison [5]. What matters here is the consequence for design: Chapter 3 describes a three-arm comparison, an unconstrained baseline, a control arm that carries the cost critic with its multiplier held at zero, and the fully constrained agent, each trained on multiple independent seeds, so that the effect of the constraint can be separated from the effect of the extra network and reported with its seed-to-seed spread rather than as a single number.

The work also continues a line of manipulator research from the same laboratory. The preceding study [10] addressed the perception and motion side of manipulation, tracking and following a target with classical image-based visual servoing. This thesis addresses the control policy instead, with reinforcement learning, and adds a safety constraint that a classical visual servoing loop does not itself provide.

1.3 Objectives

The general objective of this thesis is to determine whether expressing a manipulability safety requirement as an explicit constraint, rather than as a term in the reward, changes the safety behaviour of a learned grasping policy on a UR5e manipulator, and whether it changes how predictable that behaviour is across independent training runs.

The specific objectives are as follows:

- To implement an unconstrained PPO baseline and a constrained PPO-Lagrangian agent for a UR5e pick-and-lift grasping task in Isaac Lab, trained under a shared floor on manipulability.
- To isolate the effect of the constraint from the effect of the cost critic that implements it, by training a control arm that carries the cost critic with its Lagrange multiplier held at zero throughout training.
- To train every arm on multiple independent random seeds and report task performance and safety outcomes as a distribution across seeds, rather than as a single mean.
- To evaluate every trained policy on frozen, held-out episodes, so that the reported safety and task figures describe the policy that would actually be deployed rather than a still-learning one.

1.4 Scope

This thesis was registered under the title *Safe Adaptive Image-Based Visual Servoing with Constrained Reinforcement Learning for Precision Grasping on a UR5 Manipulator: From Simulation to Real Hardware*, and was proposed as a three-layer programme, not as Layer 1 alone.

Layer 1, delivered here, is safe reinforcement learning for grasping in simulation, using privileged pose information rather than vision. Layer 2 would have closed the loop with vision: an image-based visual servoing scheme in which the image Jacobian relating pixel motion to joint motion is tuned by the learned policy rather than fixed in advance, replacing the privileged pose feedback of Layer 1 with an eye-in-hand RGB-D camera and object detection. It would have been a direct upgrade of the predecessor project in this laboratory, which used a fixed, hand-derived image Jacobian for the same visual-servoing task [10]. Layer 3 would have carried the resulting controller onto the physical UR5e over ROS 2, closing the gap between the Robotiq 2F-85 gripper modelled in simulation and the different gripper fitted to the laboratory's real arm.

Only Layer 1 is delivered. Layers 2 and 3 are not attempted, and that is stated here as a deliberate narrowing of scope, decided against a fixed submission date, rather than as an unmet target. Building an evaluation protocol able to separate a safety constraint's effect from an implementation artefact, and to characterise it across independent seeds, could not be done to a defensible standard alongside two further systems-integration efforts of comparable size. Both dropped layers are carried forward as future work in Chapter 6.

Layer 1 on its own is not a narrow question. Constraining a policy's behaviour with an explicit safety budget, rather than through a shaped reward, is already active practice on UR-class hardware doing collision avoidance around a moving person [6], and on other manipulator platforms doing constrained grasping specifically [5]. What this thesis contributes sits inside that same practice: a comparison protocol for measuring what the constraint itself does, isolated from an implementation artefact and reported across seeds rather than as a single run, on a platform common enough in both industry and the published literature that the result should say something beyond this one laboratory's copy of it.

Within Layer 1, the scope is:

- **Task.** Single-object pick-and-lift grasping on a simulated UR5e in Isaac Lab, on the weld-grasp environment described in Chapter 3. The gripper is present and driven, but the moment of contact is abstracted rather than modelled as a full physical grasp.
- **Comparison.** Three algorithm arms: an unconstrained PPO baseline and two PPO-Lagrangian variants at different cost budgets, each trained on five independent random seeds.

- **Safety quantity.** A single constrained quantity, a floor on manipulability. Joint-limit proximity and workspace clearance are measured and reported but are not themselves constrained.
- **Evaluation.** The frozen policy produced by each run, assessed on held-out episodes with the multiplier and exploration noise fixed, not the training-time behaviour of a policy still being updated.
- **Algorithms.** The comparison uses established methods, PPO and PPO-Lagrangian; it does not propose a new constrained-RL algorithm. The contribution claimed is the comparison protocol and what it finds, not a new optimiser.
- **Hardware validation.** All results are produced in simulation. No policy from this thesis is run on the physical UR5e. That step belongs to Layer 3, out of scope here and named as future work in Chapter 6 rather than as a claim already tested.

CHAPTER 2

LITERATURE REVIEW

2.1 How to read this chapter

This chapter has one argument to make and eleven sections in which to make it. The argument is that a safety requirement on a learned manipulation policy is better expressed as a constraint with a stated budget than as a penalty term inside the reward, and that the published work closest to this thesis leaves two questions about that claim unanswered. Everything in the chapter is arranged to support, qualify or position that argument.

The chapter is organised in four movements. The first, Sections 2.2 to 2.5, builds the machinery: why reward shaping is an awkward vehicle for safety, what a constrained Markov decision process is, and which of the deep constrained algorithms this thesis uses and why. The second, Sections 2.6 to 2.8, turns to robotics and to the experimental practice of the field, and sets the reviewed studies side by side so their methods can be compared directly instead of one after another. The third, Sections 2.9 and 2.10, treats the two quantities on which this thesis actually turns: the manipulability measure that defines its safety constraint, and the seed-to-seed variance that defines how its results must be reported. The fourth, Sections 2.11 and 2.12, states the gap and closes.

A reader with limited time can take three routes. To understand *why* the thesis is constrained rather than shaped, read Sections 2.2, 2.4 and 2.9. To understand *what* was measured and against what precedent, read Sections 2.8 and 2.10. To understand *what is new*, read Section 2.11 alone, which is the section Chapter 1 refers back to and the one Chapter 4 answers.

The findings the chapter arrives at are collected below. They are stated here so that the sections that follow can be read as evidence for them instead of as a survey in search of a conclusion, and each is picked up again in Section 2.12.

Key findings of this chapter

1. A penalty weight and a cost budget are not two spellings of the same instruction. The weight fixes an exchange rate between safety and task progress that the designer must guess in advance; the budget states a limit in the units of the hazard and can be audited after training [11].
2. The constrained Markov decision process is the prevailing formalism for this in contemporary safe reinforcement learning [12], and its Lagrangian relaxation turns the

Table 2.1. Section map for this chapter, with the source that carries each section.

§	Subject	Principal sources
2.2	Why safety resists reward shaping	[11], [2]
2.3	Safe RL as a field, and its taxonomy	[1], [12]
2.4	The constrained Markov decision process	[13]
2.5	Deep constrained policy optimisation	[14], [11], [2], [7]
2.6	Safety in robot learning	[15], [16]
2.7	Deep RL on manipulators and on the UR5	[17], [8], [6]
2.8	Experimental methods of the reviewed studies	all nine experimental papers
2.9	Manipulability and singularity avoidance	[18], [3]
2.10	Reproducibility and seed variance	[4]
2.11	Research gap and positioning	[5]

guessed weight into a dual variable that the optimiser sets from the observed violation [13].

3. PPO-Lagrangian is preferred to CPO on published evidence rather than on convenience. On the eighteen-environment Safety Gym slate, CPO left a normalised violation of 0.593 against 0.026 for PPO-Lagrangian at comparable cost rates [2].
4. The Lagrange multiplier under plain dual ascent behaves as an integral controller and oscillates against the cost with a quarter-cycle lag [7]. Any multiplier trajectory reported in Chapter 4 must be read as that of a controller, not of a converged constant.
5. Yoshikawa's measure $w = \sqrt{\det(JJ^\top)}$ is the volume of the manipulability ellipsoid and the product of the Jacobian's singular values [18]. Prior learning work places it in the reward with a hand-set weight, in one case $\lambda_3 = 0.05$ [3]. This thesis places it in a constraint instead.
6. Random seed alone can produce statistically distinct outcome distributions for one algorithm on one task, at $t = -9.0916$, $p = 0.0016$ over two groups of five runs [4]. Reported means over three seeds, which is common practice in this literature, are therefore weak evidence for a safety claim.
7. The closest published comparison of PPO against its constrained variant on a manipulator [5] reports neither a control arm isolating the cost critic from the constraint nor any seed-to-seed dispersion. These are the two gaps this thesis fills.

2.2 The safety requirement in learned manipulation

A robot manipulator that learns its own control policy poses a problem that a manipulator running a hand-written controller does not. The behaviour of a classical controller can be inspected, bounded and argued about before the machine is switched on. The behaviour of a policy obtained by reinforcement learning is a consequence of an optimisation run, and is known only through the statistics of the trajectories it produces. When such a policy is to be deployed on a physical arm sharing its workspace with people, equipment or the workpiece itself, the question of what it does in the configurations it visits only rarely is not a secondary concern. It is the condition on which deployment depends at all.

The difficulty is that reinforcement learning, in its standard form, offers a single channel through which a designer expresses what the agent should and should not do: the reward function. Safety requirements expressed through that channel become penalty terms, and a penalty term must be given a weight. That weight is a trade-off rate between task performance and safety which the designer must fix before knowing what either will be worth, and which has no natural units. It is a number tuned until the resulting behaviour looks acceptable. Achiam *et al.* [11] make this argument directly in motivating constrained policy optimisation: for systems that interact physically with or around humans, it is more natural to specify a reward function *and* a set of constraints than to encode both into one scalar objective.

Ray *et al.* [2] put the same point in operational terms. If the designer selects a penalty that is too small the agent learns unsafe behaviour, and if the penalty is too severe the agent may fail to learn anything at all. A fixed trade-off, even a well-chosen one, also has no way of responding to the fact that the pressure the reward exerts on the cost changes as the policy improves. The weight that is correct early in training is not the weight that is correct late in it.

The alternative is to keep the safety requirement outside the reward and impose it as a constraint with a budget. A budget has a property the weight does not: it can be stated in advance, in the units of the quantity being constrained, and it can be audited afterwards against what the trained policy actually spent. This distinction, that *a shaped reward requires the designer to guess a weight while a constraint requires only that they state a limit*, is the premise of this thesis. Section 2.9 shows that it separates two otherwise similar bodies of work on singularity avoidance, and that the guessed weight in the nearest such study is a bare 0.05 with no stated derivation.

2.3 Safe reinforcement learning

Safe reinforcement learning is the study of methods that optimise a policy while respecting some notion of acceptable behaviour during learning, at deployment, or both. The organising survey of the field is that of García and Fernández [1], which divides the literature into two branches according to where the safety requirement is inserted. The first modifies the *optimisation criterion*, so that the objective the agent maximises is itself changed: by a risk-sensitive transformation, by a worst-case criterion, or by the addition of explicit constraints. The second modifies the *exploration process*, leaving the objective intact but restricting or guiding the actions the agent may try, typically using external knowledge such as a teacher, a demonstration set or a backup controller.

This thesis belongs to the first branch, and to its constrained-criterion sub-family in particular. The distinction matters for reading Chapter 4: methods in the second branch aim to make the learning process itself safe and are evaluated on violations accumulated *during* training, whereas methods in the first aim to produce a policy that satisfies a requirement once trained, and are properly evaluated on the frozen policy. The evaluation protocol of this work follows the latter convention, and Section 4.1 explains why an earlier version of these results, which read safety figures from training-time telemetry, was withdrawn.

Table 2.2. Where the safety requirement is inserted: the two branches of safe reinforcement learning, after [1].

Branch	**Mechanism**	**Evaluated on**
Modify the optimisation criterion	Risk-sensitive or worst-case objectives; explicit constraints on expected cumulative cost	The trained policy, on held-out episodes
Modify the exploration process	Teacher guidance, demonstrations, backup controllers, action shielding	Violations accumulated during training

The more recent review by Gu *et al.* [12] surveys the field after the deep-learning era and organises it around five open problems, which the authors abbreviate as 2H3W: how policy optimisation can search for a safe policy, how much data a safe policy costs, what the state of safe reinforcement learning applications is, what benchmarks allow fair comparison, and what the outstanding challenges are. Two of those five bear directly on this thesis. The benchmark question is why Chapter 4 reports episodic cost against a declared budget rather than an ad hoc safety score, and the applications question is what Section 2.7 answers for manipulators specifically. The same review reports that the constrained-criterion approach, expressed as a constrained Markov decision process, has become the prevailing formalism for safe reinforcement learning. That observation places the present work inside the main-

stream of the field and not at its margin, and it is why the CMDP is adopted here without further argument.

2.4 The constrained Markov decision process

The formal object underlying the constrained-criterion branch is the constrained Markov decision process, developed in its modern form by Altman [13]. A standard Markov decision process is the tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ over a state space $\mathcal{S}$, an action space $\mathcal{A}$, a transition kernel P, a reward function r and a discount factor γ. A CMDP augments this with a cost function c and a budget d, and asks for

$$\max_{\pi} \; J_r(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t)\right] \qquad \text{subject to} \qquad J_c(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{T} c(s_t, a_t)\right] \le d. \tag{2.1}$$

Chapter 3 states the problem again in the notation of the implementation. What matters here is what the formalism buys.

Two properties are relevant. The first is that the safety requirement is expressed in the units of the cost rather than in units of reward, so it can be set from a measurement of the system rather than from a tuning exercise. The calibration procedure of Section 3.7 depends on this. The second is that the constrained problem admits a Lagrangian relaxation, in which the constraint is folded into the objective through a multiplier λ that is itself optimised:

$$\max_{\theta} \min_{\lambda \ge 0} \; \mathcal{L}(\theta, \lambda) = J_r(\pi_\theta) - \lambda \big(J_c(\pi_\theta) - d\big). \tag{2.2}$$

The duality theory licensing that treatment is the contribution of [13], and it is what makes the algorithms of the next section possible. Rather than solving a constrained problem directly, they solve a sequence of unconstrained problems in which the penalty weight is not guessed by the designer but driven by the observed violation.

That last point deserves emphasis, because it is easily lost. A Lagrangian method does end up optimising a weighted sum of reward and cost, which superficially resembles reward shaping. The difference is that the weight is not a hyperparameter. It is a dual variable that rises while the budget is exceeded and falls once it is met, so the designer specifies the budget and the optimiser discovers whatever weight is required to satisfy it. A useful way to hold the two apart is to ask what the designer would have to change if the hazard became twice as costly to the operator. Under reward shaping they would have to retune a weight and retrain until the behaviour looked right. Under a constraint they would halve the budget and let the optimiser find the weight.

2.5 Constrained policy optimisation in deep reinforcement learning

The policy-gradient algorithm used as the baseline throughout this thesis is proximal policy optimisation [14], which limits the size of each policy update through the clipped surrogate objective

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t(\theta)\hat{A}_t,\ \mathrm{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\big)\Big], \qquad \rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)}, \tag{2.3}$$

where $\hat{A}_t$ is the estimated advantage and ε the clip range. PPO has become the default on-policy method for continuous control. Its popularity is partly a matter of robustness: it attains much of the reliability of trust-region methods at a fraction of their implementation complexity, which is also why it is the algorithm most commonly extended when a constraint is to be added. Both UR5 grasping studies reviewed in Section 2.7 use it, at $\varepsilon = 0.2$ in one case [8], and so does the constrained work nearest this thesis [5].

Constrained policy optimisation (CPO) [11] was the first algorithm to bring the CMDP into deep reinforcement learning at scale. It replaces the unconstrained trust-region update with

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} J_r(\pi) \quad \text{s.t.} \quad J_c(\pi) \le d, \quad D(\pi, \pi_k) \le \delta, \tag{2.4}$$

solving that problem approximately at every step so that each update both improves the return and maintains near-constraint satisfaction, with a penalty coefficient recomputed from scratch each time. Its significance for this thesis is as much conceptual as algorithmic. It established the constrained formulation as a practical alternative to reward shaping for high-dimensional continuous control, and it supplies the motivating argument quoted in Section 2.2.

The algorithm actually implemented here, however, is the Lagrangian variant of PPO, as specified in the Safety Gym benchmark report of Ray *et al.* [2]. That report proposes constrained reinforcement learning as the standard formalism for safe exploration and supplies a benchmark suite of continuous-control environments together with reference implementations of PPO, TRPO, their Lagrangian forms, and CPO. Its constrained agents are trained by gradient ascent on θ and descent on λ in Equation (2.2), at a cost limit of $d = 25$ throughout, over episodes of 1000 steps and three random seeds per environment.

Two of its findings bear directly on the design of this study. The first is the specification itself: the constrained agent studied here is PPO-Lagrangian as defined in [2], in which an adaptive penalty on the safety cost is maintained by dual ascent on a Lagrange multiplier. The budget of 25 used throughout Chapter 4 is inherited from that reference configuration

and was not chosen freshly, which is worth stating because it removes one degree of freedom from the present design. The second finding is empirical, and is the reason PPO-Lagrangian was preferred to CPO. Table 2.3 reproduces the relevant comparison.

Table 2.3. Normalised end-of-training metrics on the eighteen-environment Safety Gym slate, averaged over three random seeds per environment, from [2]. Values are normalised against unconstrained PPO, so 1.0 is the unconstrained baseline and lower is safer. $\bar{M}_c$ is the constraint violation and $\bar{\rho}_c$ the cost rate.

Algorithm	**Return** $\bar{J}_r$	**Violation** $\bar{M}_c$	**Cost rate** $\bar{\rho}_c$
PPO (unconstrained)	1.000	1.000	1.000
TRPO (unconstrained)	1.094	1.132	1.004
PPO-Lagrangian	**0.240**	**0.026**	**0.245**
TRPO-Lagrangian	0.331	0.018	0.265
CPO	0.784	0.593	0.646

CPO retains far more return than either Lagrangian method, but it leaves a normalised violation of 0.593, which is to say that it removes only about forty percent of the unconstrained agent's violations. The Lagrangian methods remove roughly ninety-seven percent of them. Ray *et al.* attribute the shortfall to approximation error in CPO's analytic step and note that the result contradicts the one reported in [11] on easier environments. Given that, adopting CPO would have required a justification this work does not have. The Lagrangian form is both simpler to implement correctly and better supported by the published comparison.

Lagrangian methods carry a characteristic weakness, and this thesis encounters it directly in Chapter 4. Stooke *et al.* [7] identify the cause precisely: the plain dual-ascent update of λ accumulates the running constraint violation, which is integral control on the constraint and nothing else. An integral controller with no proportional or derivative action overshoots, and the cost and the multiplier consequently oscillate against one another with a phase lag of about a quarter cycle. Figure 2.1 is their demonstration of this.

Their remedy is to treat the multiplier as a control input and apply full PID control to it,

$$\Delta_k = J_c(\pi_k) - d, \qquad I_k = \big(I_{k-1} + \Delta_k\big)_+, \qquad \partial_k = \big(J_c(\pi_k) - J_c(\pi_{k-1})\big)_+, \tag{2.5}$$

$$\lambda_k = \big(K_P\Delta_k + K_I I_k + K_D \partial_k\big)_+, \tag{2.6}$$

where $(\cdot)_+$ denotes projection onto the non-negative reals, and where setting $K_P = K_D = 0$ recovers the traditional Lagrangian method exactly. Two further details of their implementation are inherited by this thesis. They maintain a *separate* cost-value approximator alongside the reward-value approximator rather than folding $r + \lambda c$ into a single critic, on the grounds that λ may change quickly; and they rescale the policy gradient by $1/(1+\lambda)$ so that a large multiplier does not enlarge the effective step size. The first of those two is the reason the con-

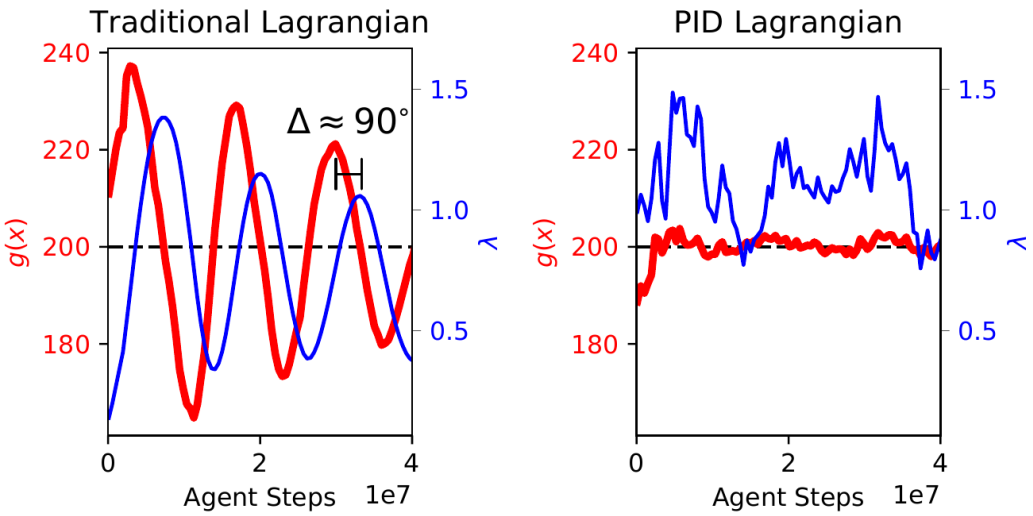

Figure 2.1. Left: under the traditional Lagrangian update the constraint function $g(x)$ and the multiplier λ oscillate about the cost limit with a phase shift of about 90°, the signature of an ill-tuned integral controller. Right: adding proportional and derivative terms damps the oscillation and holds the cost at the limit. Environment DOGGOBUTTON1, cost limit 200. Reproduced from Stooke *et al.* [7], Figure 1.

strained agent in this study carries an additional network that its baseline does not, which in turn is the reason a control arm is needed at all. Section 2.11 returns to this. Their analysis of multiplier dynamics is used in Section 4.6 to interpret the values observed at the end of training, and the PID form itself is identified in Chapter 6 as the natural extension of this work. It is also worth noting that [7] names its PPO-based agent CPPO, the same abbreviation this thesis uses for its constrained arm.

The benchmark suites have since been superseded. Safety Gymnasium [19] supersedes the original Safety Gym environments and ships alongside a library of safe reinforcement learning algorithms under a common evaluation protocol. Its reporting conventions, episodic cost against a stated budget and violation rates measured per episode, are those adopted in Chapter 4.

2.6 Safety in robot learning

The literature reviewed so far approaches safety from the machine-learning side. Robotics approaches it from the control side, with a vocabulary of invariant sets, barrier certificates and stability guarantees that developed independently. Brunke *et al.* [15] survey the intersection and, more usefully for the present purpose, reconcile the two vocabularies into a common framework spanning learning-based control through to safe reinforcement learning.

That reconciliation is necessary here because this thesis sits across the boundary. The quantities constrained in this work (the manipulability measure, the distance to a joint limit, the clearance above the work surface) are control-theoretic constructs describing the kinematic

state of the arm. The mechanism enforcing them is a learning-theoretic one: a cost critic and a dual variable. Neither literature alone supplies the language for that combination.

The survey also draws a distinction that constrains what may be claimed from the results of this work. Methods enforcing a constraint in expectation, as a Lagrangian method does, bound *expected* exposure to unsafe configurations; they do not bound the severity of any individual excursion. Methods supplying a safety certificate, such as a control barrier function acting as a filter on the commanded action, offer the latter guarantee but require a model. This distinction is returned to in Section 4.5, where a counter-result of exactly this shape is reported, and in Chapter 6, where the two mechanisms are argued to address different halves of the problem.

Within manufacturing specifically, Elguea-Aguinaco *et al.* [16] review reinforcement learning for contact-rich manipulation from 2017 onward, covering reward design, sim-to-real transfer and the treatment of safety. Their survey is used in Chapter 3 to establish which of the design choices made in this work follow established practice.

2.7 Deep reinforcement learning for robotic manipulation

Applications of deep reinforcement learning to manipulation divide broadly by algorithm family. Shahid *et al.* [17] compare an on-policy method (PPO) against an off-policy method (soft actor-critic) on a grasping task with a Franka Emika Panda in MuJoCo. Their task is split into a reaching phase and a lifting phase with a separate dense reward for each, built from a distance term, an end-effector velocity term and a gripper-opening term, to which penalties on joint-position proximity, joint acceleration and obstacle approach are added by fine-tuning rather than all at once. The policy runs at 250 Hz against a 500 Hz joint controller, with linear interpolation between successive policy actions, and is trained for 10^7 steps in episodes of 600 steps. Evaluation on the physical Panda, with no real-world training data, gives ten successes out of ten on each of four objects: the nominal 6 cm cube, a smaller 4 cm cube, a cylinder and a screwdriver.

That study is relevant here in three ways. It is the methodological template for an on-policy against off-policy comparison on a manipulation task; it demonstrates zero-shot transfer of the learned behaviour to a physical robot, which supports the position taken in Chapter 6 that results of this kind are transferable in principle; and it is the precedent for the off-policy arm that was planned for this study and ultimately not trained, as recorded in Section 4.7.

Work specific to the UR5 platform is less common. Ferreira and Barbosa [8] train a UR5 fitted with a Robotiq 2F-85 gripper to grasp randomly positioned objects in a PyBullet environment simulated at 240 Hz, comparing soft actor-critic against PPO under identical conditions with domain randomisation. Their scene is reproduced in Figure 2.2, and is the closest published analogue to the environment of this thesis in both robot and gripper.

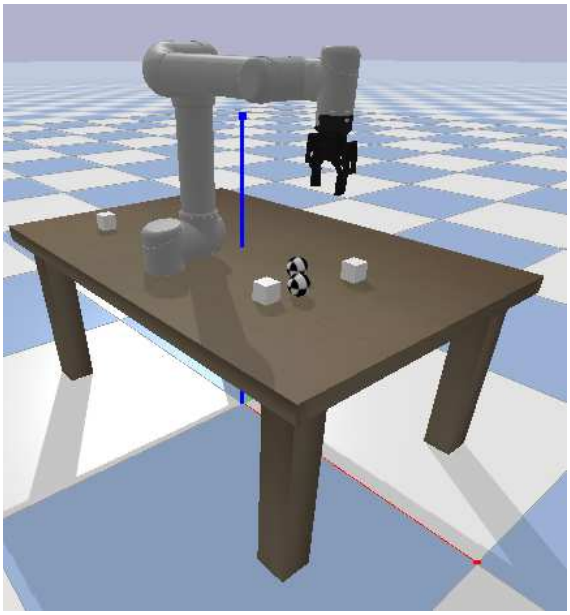

Figure 2.2. The `UR5GraspingEnv` PyBullet environment: a UR5 arm with a Robotiq 2F-85 gripper and randomised objects on the work surface. This is the closest published analogue to the simulation environment of this thesis in both manipulator and end-effector. Reproduced from Ferreira and Barbosa [8], Figure 1, under CC BY 4.0.

Their reward is shaped, with the form

$$R(s,a) = -\lambda \lVert p_g - p_o \rVert_2 + r_s \mathbb{I}_{\text{grasp}} + r_p \mathbb{I}_{\text{pose}}, \tag{2.7}$$

with $\lambda = 0.1$ weighting the gripper-to-object distance penalty, $r_s = 10$ for a grasp held at least 0.5 s and $r_p = 5$ for correct placement orientation. Their own ablation shows how much of the result rests on those hand-set numbers: removing the distance term costs ten percentage points of success, removing the grasp reward twelve, removing the pose reward eight, and reducing the whole function to its two sparse indicators costs eight. Both agents were trained for 100,000 timesteps at a learning rate of 3×10^{-4} with a batch size of 64 and a two-layer 256-unit MLP. They report grasp success of 87 % for SAC and 82 % for PPO, with post-grasp placement success of 75 % and 68 % respectively.

Those are the closest published figures for a learned UR5 grasping policy, and they are quoted here with one qualification that must be carried into any comparison. Their task requires a full physical grasp, in which the fingers close on the object and the resulting contact must be stable, whereas the task studied in this thesis abstracts the grasp as a weld for the reasons given in Section 3.3. The success rates of Chapter 4 are therefore not directly comparable to theirs, and are not presented as though they were.

On the safety side, Xia *et al.* [6] apply deep reinforcement learning to proactive obstacle avoidance for a UR5e in a human-robot collaborative assembly setting, using an RGB-D

camera for environmental perception and a wrist force-torque sensor, and training a soft actor-critic agent under a composite reward. They report a 93 % success rate in avoiding dynamic obstacles while still reaching the goal pose. Their work establishes that safe operation of this specific manipulator is an active industrial concern and not an academic construction, which is the premise on which the practical relevance of this thesis rests. It also illustrates the alternative design route: where this thesis expresses safety as a budget on a kinematic cost, Xia *et al.* express it as another term inside a composite reward, and the 93 % they report is a success rate rather than a bound on anything.

2.8 Experimental methods of the reviewed studies

Describing each study in turn, as the previous sections do, makes it hard to see what the field holds fixed and what it varies. Table 2.4 therefore sets the experimental apparatus of the reviewed work side by side. It covers only the papers that report their own experiments; the three surveys are excluded, since they report other people's.

Table 2.4. Experimental apparatus of the reviewed studies. Seed counts are as reported by the authors; a dash means the paper does not state one. All values were read from the source papers, which are held in `Thesis_LaTeX/source_papers/`.

Study	Platform	Simulator	Algorithms	Seeds	Headline result
Ray *et al.* [2]	Point, Car, Doggo	MuJoCo / Safety Gym	PPO, TRPO, both Lagrangian, CPO	3	PPO-Lag. violation 0.026 vs CPO 0.593 (normalised)
Stooke *et al.* [7]	Doggo	Safety Gym	CPPO with PID multiplier	–	Cost oscillation damped; violations reduced
Henderson *et al.* [4]	MuJoCo locomotion	OpenAI Gym	TRPO, PPO, DDPG, ACKTR	5 (and 10)	Seed alone gives distinct distributions, $p = 0.0016$
Shahid *et al.* [17]	Franka Panda	MuJoCo	PPO, SAC	–	100 % grasp success, zero-shot on hardware
Ferreira and Barbosa [8]	UR5 + 2F-85	PyBullet	PPO, SAC	–	SAC 87 % vs PPO 82 % grasp success
Xia *et al.* [6]	UR5e	Custom + ROS	SAC	–	93 % dynamic obstacle avoidance
Shen *et al.* [3]	4-DOF planar	Custom	SAC in Jacobian null space	5	96.8 % vs 77.4 % avoidance; higher mean w
Khan *et al.* [5]	Franka Panda	Safety Gym	PPO, cPPO	–	cPPO cost 12.32 vs PPO 17.76 (SOPM)
This thesis	**UR5e**	**Isaac Sim**	**PPO, control arm, cPPO**	**5**	**Chapter 4**

Three patterns are visible in that table, and each shapes a decision made in Chapter 3.

The first is that the cost limit $d = 25$ recurs. Ray *et al.* set it as the reference configuration for Safety Gym [2]; Khan *et al.* inherit it unchanged along with $\gamma = 0.99$, $\lambda_{\mathrm{GAE}} = 0.97$, a penalty learning rate of 5×10^{-2} and a penalty initialised at 1.0 [5]; and this thesis inherits it in turn. A number reused across three studies is not thereby correct, but it does mean the budget in Chapter 4 is a convention and not a free parameter tuned to flatter the result.

The second is that network capacity in this literature is small. Ray *et al.* use two hidden layers of 256 units, Ferreira and Barbosa the same, and Khan *et al.* two layers of 64 units with an eight-dimensional observation and a four-dimensional action. None of the reported outcomes appear to be limited by the size of the function approximator, which is a useful thing to know before attributing any difference in this thesis to architecture.

The third is the one that matters most, and it is a pattern of absence. Of the eight experimental studies in the table, three report a seed count and five do not. None of the three that do report per-seed dispersion; they report a mean, or a shaded band whose composition is not decomposed. Section 2.10 takes up what follows from that.

2.9 Manipulability and singularity avoidance

The operative safety constraint in this thesis is a floor on manipulability. This section develops the measure carefully, because it is the single quantity on which the safety claim of this work rests, and because the difference between this thesis and its nearest neighbour in the literature is entirely a difference in where that quantity is placed.

2.9.1 The measure and what it means

Consider a manipulator with n joints, joint vector $\theta \in \mathbb{R}^n$, whose task is described by $m \leq n$ manipulation variables $\mathbf{r} \in \mathbb{R}^m$. Differentiating the forward kinematics gives the velocity relation

$$\dot{\mathbf{r}} = J(\theta)\,\dot{\theta}, \qquad J(\theta) \in \mathbb{R}^{m \times n}, \tag{2.8}$$

where J is the manipulator Jacobian. Yoshikawa [18] defines a configuration θ^* to be *singular* when $\operatorname{rank} J(\theta^*) < m$. At such a configuration the end-effector cannot be moved instantaneously along at least one Cartesian direction, no matter what joint velocities are commanded, and the manipulator has lost part of its ability to do its task. As a scalar index of how far a configuration is from that condition he proposes

$$w(\theta) = \sqrt{\det\!\big(J(\theta)J(\theta)^{\top}\big)}. \tag{2.9}$$

Three properties established in [18] explain why this particular combination is the right one, and each of them is used somewhere in this thesis.

It is the product of the singular values. Writing the singular value decomposition $J = U\Sigma V^\top$ with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_m \geq 0$,

$$w = \sigma_1 \sigma_2 \cdots \sigma_m. \tag{2.10}$$

Because w is a product, it goes to zero as soon as any single σ_i does. It is therefore sensitive to the loss of *one* direction of motion even when the arm remains perfectly capable in every other direction, which is exactly the failure that matters.

It is the volume of the manipulability ellipsoid. The set of end-effector velocities reachable from unit-norm joint velocities, $\{\dot{\mathbf{r}} = J\dot{\theta} : \|\dot{\theta}\| \leq 1\}$, is an ellipsoid whose principal axes are $\sigma_1 u_1, \ldots, \sigma_m u_m$, and whose volume is proportional to w. The index is thus not an abstract conditioning number but a measure of how much end-effector motion a unit of joint effort actually buys.

It reduces to $|\det J|$ for a non-redundant arm. When $m = n$, Equation (2.9) simplifies to $w = |\det J|$. The UR5e used in this work has six joints controlling a six-dimensional end-effector pose, so this is the form that applies here.

Yoshikawa also gives the simplest worked example, and it is worth carrying because it makes the quantity concrete. For a planar two-link arm with link lengths ℓ_1 and ℓ_2 and joint angles θ_1, θ_2, the measure evaluates to

$$w = \ell_1 \ell_2 |\sin \theta_2|, \tag{2.11}$$

which is maximised at $\theta_2 = \pm 90^\circ$ and vanishes when the arm is fully extended or fully folded. The same result holds for the SCARA-type manipulator. Yoshikawa notes that a human arm, treated as a two-link mechanism, satisfies $\ell_1 \approx \ell_2$ and is habitually used with the elbow near a right angle, so people appear to adopt high-manipulability postures without being told to. The intuition to keep is that a straightened arm is a weak arm: it can still push along its own length, but it can barely move sideways, and a controller asking it to do so will demand very large joint rates.

That last consequence is the practical reason a floor on w is a safety constraint and not merely a performance one. Near a singularity the inverse mapping is ill-conditioned, so a small commanded Cartesian velocity maps to a large joint velocity. On a real UR5e whose joints are capped at $180^\circ \mathrm{s}^{-1}$, that appears either as a violent motion or as a protective stop. Yoshikawa anticipates this by normalising the Jacobian by the per-joint maximum velocities $\dot{\theta}_{i0}$ when those differ, which for the UR5e they do not, since all six joints share the same

ceiling. The unnormalised form of Equation (2.9) is therefore used directly in Chapter 3, where the cost term is defined as an excursion below a floor $w_{\min}$, and where Section 3.7 records the recalibration of that floor from 0.045 to 0.06 after the baseline policy's natural operating distribution was measured.

2.9.2 The same measure, placed differently

Singularity avoidance has been treated within learning frameworks before, and the manner of that treatment is precisely where this thesis differs. Shen *et al.* [3] address reactive obstacle avoidance for redundant manipulators by learning self-motion in the null space of the Jacobian, so that the commanded task velocity is preserved exactly while the learned action redistributes the remaining freedom:

$$\dot{q}_N = \left(I - J^{\dagger}J\right)\phi, \qquad \dot{q}_D = J^{\dagger}\dot{x}_D, \tag{2.12}$$

with ϕ the learned action and $J^{\dagger}$ the pseudo-inverse. Their agent is soft actor-critic, and their reward is the weighted sum

$$r = \lambda_1 r_o + \lambda_2 r_a + \lambda_3 r_m, \qquad r_o = \sum_{i=1}^{n} \min\left(\frac{\|d_i\|}{d_s} - 1, 0\right), \qquad r_m = \sqrt{\det\left(JJ^{\top}\right)}, \tag{2.13}$$

in which r_o penalises obstacle proximity inside a safe distance d_s, r_a penalises null-space joint motion, and r_m is Yoshikawa's measure entered directly as a reward term. Their reported weights are $\lambda_1 = 1$, $\lambda_2 = 0.2$ and $\lambda_3 = 0.05$, and a collision or joint-limit breach triggers a flat penalty of -10. The method works: over 1000 randomly generated obstacle configurations it raises avoidance success from 77.4 % to 96.8 % against a gradient-projection baseline in the two-obstacle scenario, while holding a higher mean manipulability throughout (Table 2.5), and it computes the null-space motion faster because a forward pass replaces an optimisation.

Figure 2.3 shows what that improvement looks like in configuration space. The contours are the manipulability landscape over two joint angles, and the two short traces are the paths taken by the baseline and by the learned policy from the same start. The learned policy moves towards higher manipulability while the baseline does not.

The objective pursued in [3] is the same as the one pursued here, and the quantity is identical down to the square root. The mechanism is not. Placing manipulability in the reward requires a weight relating a unit of w to a unit of task progress, and $\lambda_3 = 0.05$ is that weight. Nothing in the paper derives it, and nothing could, because there is no physical exchange rate between an ellipsoid volume and a metre of end-effector travel. It is a number that produced

Table 2.5. Reported comparison of the gradient projection method (GPM) against the reinforcement learning method of Shen *et al.* [3], over 1000 randomly generated obstacle configurations. Scenario I has one obstacle, Scenario II has two. $\bar{m}$ is the average manipulability under successful avoidance.

Measure	GPM	RL (null space)
Success rate, Scenario I	100 %	100 %
Success rate, Scenario II	77.4 %	**96.8 %**
$\bar{m}$, Scenario I	3.78	**3.95**
$\bar{m}$, Scenario II	3.63	**3.72**
Null-space solve time, Scenario I	1.484 ms	**1.155 ms**
Null-space solve time, Scenario II	2.048 ms	**1.372 ms**

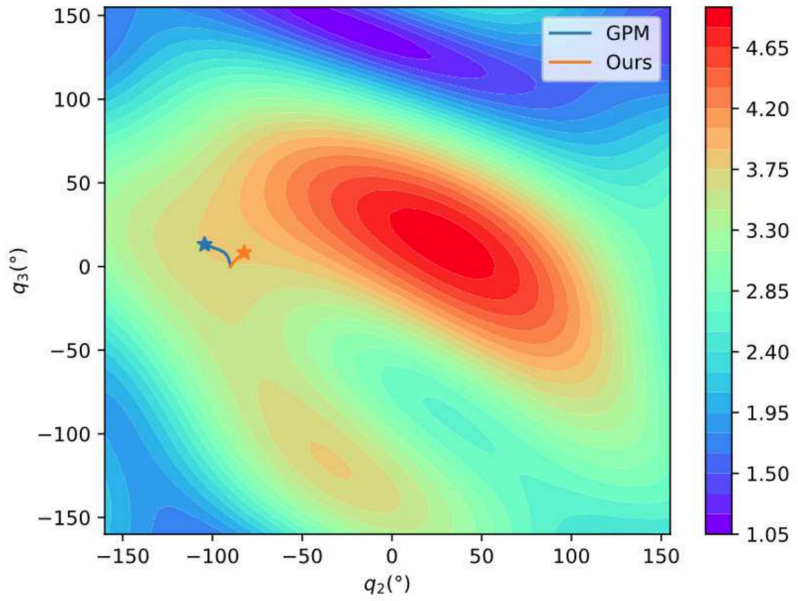

Figure 2.3. Manipulability of a 4-DOF planar manipulator over the q_2–q_3 plane, with the paths taken by the gradient projection method (GPM) and by the reinforcement learning method from a common start. The colour scale is the manipulability measure w. The learned policy moves towards the higher-manipulability region. Reproduced from Shen *et al.* [3], Figure 12, under CC BY 4.0.

acceptable behaviour. Placing the same quantity in a constraint, as this thesis does, requires instead a budget, which is a statement of how much cumulative proximity to singularity an episode may incur. That statement can be measured from a baseline policy, as Section 3.7 does, and audited against the trained one, as Section 4.5 does.

The comparison is worth stating plainly, because it is the sharpest available expression of what this thesis does differently. Both approaches seek the same behaviour from the same measure. One obtains it by tuning a trade-off; the other by declaring a limit.

2.10 Reproducibility and seed variance

A final strand of the literature is methodological instead of algorithmic, and it bears on the central experimental finding of this work. It also explains a protocol decision that would otherwise look like over-caution.

Henderson *et al.* [4] examine the reproducibility of results in deep reinforcement learning and identify several sources of variation that are not the contribution being claimed: network architecture and activation function, reward scaling, choice of environment, the particular baseline codebase used, and the random seed. Their treatment of the last is the one that matters here, and it is unusually direct. They run ten trials of TRPO on HalfCheetah-v1 under one fixed hyperparameter configuration, varying nothing but the seed, split those ten arbitrarily into two groups of five, and average each group. If seed were noise around a stable mean, the two curves would coincide. Figure 2.4 shows that they do not. A two-sample t-test across the training distribution gives $t = -9.0916$ and $p = 0.0016$: two groups drawn from the same algorithm, the same task and the same settings belong to statistically distinct distributions.

The size of the effect is easier to read from their bootstrap intervals than from the curves. Table 2.6 reproduces them. On HalfCheetah-v1, DDPG's 95 % interval runs from 3664 to 6574 about a mean of 5037, a span of more than half the mean. Any two papers reporting single runs of the same algorithm on that environment could differ by a factor approaching two without either having done anything wrong.

Table 2.6. Bootstrap mean and 95 % confidence bounds of final return, 10,000 bootstrap iterations with the pivotal method, from Henderson *et al.* [4]. The width of each interval is the spread attributable to seed and run-to-run variation, not to any difference in method.

Environment	DDPG	ACKTR	TRPO	PPO
HalfCheetah-v1	5037 (3664, 6574)	3888 (2288, 5131)	1254.5 (999, 1464)	3043 (1920, 4165)
Hopper-v1	1632 (607, 2370)	2546 (1875, 3217)	2965 (2854, 3076)	2715 (2589, 2847)
Walker2d-v1	1582 (901, 2174)	2285 (1246, 3235)	3072 (2957, 3183)	2926 (2514, 3361)
Swimmer-v1	31 (21, 46)	50 (42, 55)	214 (141, 287)	107 (101, 118)

Their recommendation follows from this: report results over multiple independent seeds with

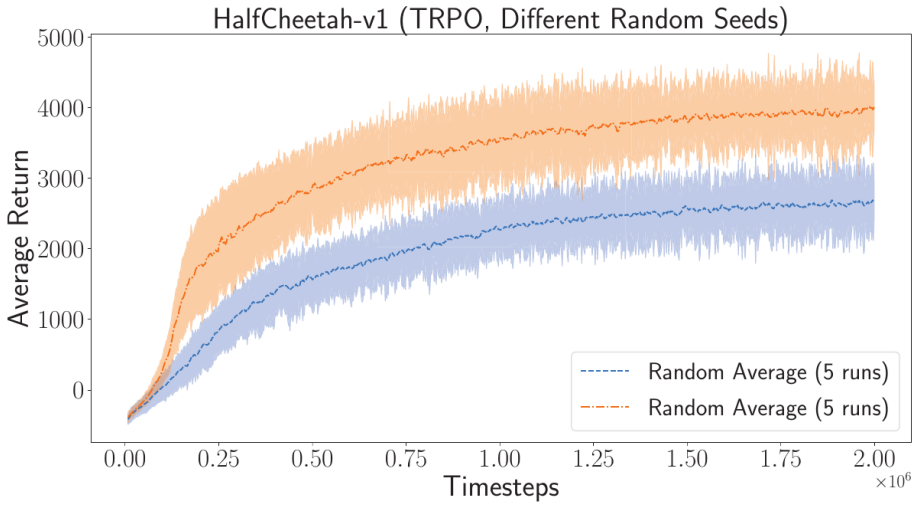

Figure 2.4. TRPO on HalfCheetah-v1 under one fixed hyperparameter configuration, averaged over two arbitrary groups of five random seeds each. Nothing differs between the two curves except which seeds fell into which group. The two-sample t-test across the training distribution gives $t = -9.0916$, $p = 0.0016$. Reproduced from Henderson *et al.* [4], Figure 5.

a measure of dispersion, treat comparisons between methods with corresponding caution, and do not select the best of several runs. They decline to name a required number of trials, observing that the appropriate count depends on how unstable the environment is, but they note that averaging fewer than five is common in published work and is not defensible.

Set that against Table 2.4. Ray *et al.* use three seeds [2]; Shen *et al.* use five and plot the minimum-to-maximum band [3]; five of the eight experimental studies reviewed here, including the one nearest to this thesis, do not state a seed count at all. This is not a criticism of any individual paper, since the convention is field-wide. It is an observation about what the published safety numbers can and cannot support.

Three consequences follow for this thesis, and they are of different kinds.

The first is procedural. The protocol of Chapter 3 trains every arm on five independent seeds (1, 3, 4, 52 and 54) and reports dispersion across them rather than a single figure. That protocol is justified by reference to [4] rather than by assertion, which is why this section sits in the literature review and not in the methodology.

The second is interpretive. The seed-to-seed spread reported in Section 4.6 is not an anomaly of this environment. It is an instance of a documented property of the method, which is why it is presented there as a measured quantity and not as a surprising discovery.

The third is about what a safety claim is for. Henderson *et al.* are concerned with fair comparison between algorithms, which is a concern of authors. An integrator deploying a policy has a different one. They do not obtain the mean over training runs they will never perform; they obtain the single policy they happened to train, on the single seed they happened to use.

For that reader, the dispersion is not an error bar around the result. It is the result. This reframing is what motivates the question Chapter 4 actually asks, which is whether a constraint tightens the spread of safety outcomes as well as shifting their centre.

2.11 Research gap and positioning

The nearest published analogue to the present study is that of Khan *et al.* [5], which compares a model-free PPO baseline against its constrained variant for precision grasping and safety-critical coordination on a robotic arm, using the Safety Gym framework extended with a Franka Panda manipulator, and introduces a tactile feedback scheme to accelerate learning under safety compliance. Their agent is a two-layer 64-unit MLP actor-critic on an eight-dimensional observation, trained for 250 epochs of 2000 steps at the Safety Gym reference settings, including the cost limit of 25 that this thesis also uses. Their reported outcome is given in Table 2.7.

Table 2.7. Reported evaluation of PPO against cPPO on two manipulation tasks, from Khan *et al.* [5]. SOPM is single-object precision manipulation, MOAM is multi-object atomic manipulation. The cost limit is 25 in both. No seed count and no dispersion are reported.

Task	Algorithm	Final avg. reward	Final avg. cost	Cumulative cost
SOPM	PPO	9.5	17.76	5954
SOPM	cPPO	8.8	**12.32**	5806
MOAM	PPO	7.5	38.80	13,880
MOAM	cPPO	7.2	**29.95**	10,919

The result is a clean one: on both tasks the constrained agent gives up a little reward and returns a substantially lower episodic cost. It is the closest match to the present work in method, in framing and in intent, and this thesis should be read as continuing that line of work rather than as departing from it.

Two gaps are nonetheless visible in that line, and it is these the present study addresses. They are stated below in the form they take as design requirements.

Gap 1. No control arm isolates the cost critic from the constraint.

A constrained agent of this family differs from its unconstrained baseline in more than the constraint. Following [7], it constructs and trains an additional cost-value network, and that network shares an optimiser, a rollout buffer and a random number stream with the machinery that trains the policy. Its presence can therefore perturb the result even when the constraint is inactive and $\lambda = 0$. Formally, the measured difference decomposes as

$$\underbrace{Q(\text{cPPO}) - Q(\text{PPO})}_{\text{what is reported}} = \underbrace{\big[Q(\text{ctrl}) - Q(\text{PPO})\big]}_{\text{implementation term}} + \underbrace{\big[Q(\text{cPPO}) - Q(\text{ctrl})\big]}_{\text{constraint term}},$$

where *ctrl* is an arm carrying the cost critic with the multiplier pinned to zero. Without that third arm the two terms are summed and cannot be separated. Published comparisons in this area, including [5], do not report one.

Why it matters here: Chapter 4 shows this is not a hypothetical concern. An earlier version of the present work reported a large advantage for the constrained agent that was later traced to exactly this class of implementation artifact and withdrawn in full.

Gap 2. No account of seed-to-seed dispersion.

Comparisons in this literature are reported as means. Five of the eight experimental studies in Table 2.4 do not state how many seeds those means cover, and none reports the per-seed values. Given [4], a mean over few seeds is weak evidence for a safety claim.

Why it matters here: the deployed quantity is not the mean over training runs that will never be performed. It is the single policy that was actually trained. Whether a constraint tightens the *spread* of safety outcomes as well as shifting their mean is therefore a question of practical consequence, and it is not one the existing literature answers.

This thesis addresses both. It trains a three-arm comparison on five independent seeds each: an unconstrained baseline, a control arm carrying the cost critic with the multiplier pinned to zero, and the constrained agent. It then reports the decomposition of the constrained-against-baseline difference into an implementation term and a constraint term, together with the dispersion of every reported quantity across seeds. Table 2.8 sets that design against the two studies it most closely resembles.

The work is also positioned locally. It continues a line of manipulator research in the same laboratory, of which the immediately preceding study [10] implemented classical image-based visual servoing with CSRT tracking on a five-degree-of-freedom manipulator. Where that work addressed the perception and servoing loop with classical methods, the present work addresses the control policy with learned ones, and adds the safety constraint that classical visual servoing does not itself provide.

2.12 Summary

The path from the first section of this chapter to the last is short enough to retrace in one pass, and doing so makes clear why the experiment of Chapter 4 takes the form it does.

It begins with a mismatch. Reinforcement learning gives a designer one channel through which to say what the agent should do, and safety requirements do not fit down it comfort-

Table 2.8. Positioning of this thesis against the two nearest published studies. A tick means the property is present and reported; a cross means it is not.

Property	**Khan *et al.*** [5]	**Shen *et al.*** [3]	**This thesis**
Manipulator platform	Franka Panda	4-DOF planar	UR5e
Safety expressed as	Constraint	Reward term	Constraint
Safety quantity	Collision cost	Manipulability w	Manipulability w, joint limit, collision
Constrained algorithm	PPO-Lagrangian	SAC (unconstrained)	PPO-Lagrangian
Cost-critic control arm	×	×	✓
Seed count stated	×	✓ (5)	✓ (5)
Per-seed dispersion reported	×	×	✓
Frozen-policy evaluation	×	✓	✓

ably. A hazard entered into the reward becomes a penalty, a penalty needs a weight, and that weight is an exchange rate between injury and task progress that nobody can derive [11, 2]. Section 2.9 produced the concrete instance: in the closest published treatment of singularity avoidance by learning, that exchange rate is $\lambda_3 = 0.05$, and the paper offers no account of where it came from [3]. That is not a lapse on the authors' part, since there is nothing available to derive it from.

The constrained Markov decision process resolves the mismatch by moving the requirement out of the objective and into a separate condition with a budget [13], an approach that Gu *et al.* [12] report has become the prevailing formalism in the field. Its Lagrangian relaxation does eventually optimise a weighted sum, but the weight is now a dual variable driven by the measured violation rather than a hyperparameter fixed in advance. Among the deep algorithms implementing this, PPO-Lagrangian was chosen over CPO on published evidence: on the Safety Gym slate, CPO removed about forty percent of the unconstrained agent's violations where the Lagrangian methods removed about ninety-seven [2]. That choice carries a known cost, which Stooke *et al.* [7] diagnose exactly. Plain dual ascent is integral control, integral control overshoots, and the multiplier and the cost consequently chase one another around the budget. Any λ reported in this thesis is a controller mid-flight, not a converged constant, and the separate cost critic that the same authors recommend is what later forces the control arm into the experimental design.

Turning to robotics narrowed the field quickly. Learned grasping on manipulators is well established, and the numbers are respectable: 100 % grasp success with zero-shot hardware transfer on a Franka Panda [17], 87 % for SAC against 82 % for PPO on a UR5 with a

Robotiq gripper [8], and 93 % dynamic obstacle avoidance on a UR5e in a human-robot cell [6]. None of them is a bound, however. They are success rates obtained under a shaped reward, and a success rate says nothing about the configurations the arm passed through on the way. The safety literature and the manipulation literature had been read separately up to that point, and Brunke *et al.* [15] supply the vocabulary that joins them, along with the caveat this thesis must respect: a constraint in expectation bounds average exposure, not the severity of any one excursion.

The operative quantity then came from an older source. Yoshikawa's manipulability measure $w = \sqrt{\det(JJ^\top)}$ is the product of the Jacobian's singular values and the volume of the velocity ellipsoid [18], which is why it collapses when the arm loses a single direction of motion instead of degrading gradually. A straightened arm has a small w, and near that condition a modest Cartesian command becomes a large joint rate, which on a real UR5e is either a violent motion or a protective stop. That is the mechanism by which a kinematic index becomes a safety quantity, and it is the reason this thesis constrains w rather than merely reporting it.

The last piece was methodological, and it changed the question instead of the method. Henderson *et al.* [4] showed that ten runs of one algorithm on one task, differing only in seed, split into two groups of five that are statistically distinct at $p = 0.0016$. Reading Table 2.4 in that light turned an incidental observation into a finding: of the eight experimental studies reviewed here, five do not state a seed count, and none reports per-seed values. The published safety numbers in this area are therefore means of unknown composition. Since a practitioner deploys one policy rather than a mean, the useful property of a safety mechanism may be how tightly it constrains the spread of outcomes rather than where it puts their centre.

Those threads converge on the two gaps of Section 2.11, both visible in the study nearest to this one [5]. The first is that no control arm separates the effect of adding a cost critic from the effect of activating the constraint, so the reported difference sums two mechanisms of different kinds. The second is that no dispersion is reported, so the reported difference is a mean over an unstated number of runs. Neither is a fault of that paper; both are field-wide conventions. But both are answerable, and answering them requires only experimental design and no new algorithm.

That is the scope this thesis takes on, and its boundaries should be stated as plainly as its content. It proposes no new optimiser: the constrained agent is PPO-Lagrangian as published [2], deliberately unmodified, because a modified algorithm would reintroduce the confound the control arm exists to remove. It works in simulation, so its claims concern a simulated UR5e and are transferable only in the sense that [17] demonstrates such transfer is possible. It abstracts the grasp as a weld, so its success rates are not comparable with the full-grasp figures of [8]. Within those boundaries it contributes a three-arm, five-seed comparison in which the implementation term and the constraint term are separated by construction and

every quantity is reported with its spread across seeds. Chapter 3 sets out the method by which that is done.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 Preliminaries

This section describes the tools this study is built from and fixes the vocabulary the rest of the chapter uses. It is deliberately descriptive. The argument for expressing a safety requirement as a constraint rather than as a reward term, and the published evidence behind the particular algorithm chosen, belong to Chapter 2 and are not restated here.

3.1.1 Isaac Sim and Isaac Lab

NVIDIA Isaac Sim is a robotics simulator built on the Omniverse platform. Scenes are described in Universal Scene Description (USD) files, which carry the link geometry, the joint definitions, the mass and inertia properties and the collision meshes of a robot in one asset, and the physics is solved by PhysX. Two properties of it matter to this work. Rigid-body dynamics, contact resolution and articulation solving all execute on the GPU, so many independent copies of a scene advance together without their state returning to the CPU between steps. And the asset that is simulated is the asset that is rendered, so a discrepancy between what the physics engine believes about the robot and what appears in the viewport is visible rather than hidden. Section 3.3 describes a gripper fault that was diagnosed for exactly that reason.

Isaac Lab is the reinforcement-learning framework layered on top of the simulator. It is manager-based: an environment is assembled by declaring configuration objects for observations, actions, rewards, terminations, events and commands, instead of by writing a monolithic step function. A task defined this way can be retargeted onto different hardware by replacing the robot articulation, the action term and the end-effector frame while the reward and termination structure of a tested environment stays untouched, which is what Section 3.3 does with Isaac Lab's Franka cube-lift task. Isaac Lab has no publication of its own; its predecessor Orbit is the citable reference [20], and the GPU-resident training design both inherit was introduced with Isaac Gym [21].

3.1.2 Reinforcement learning, on-policy and off-policy

A reinforcement-learning agent observes the state of its environment, selects an action from a policy, receives a scalar reward, and lands in a new state. Repeating this produces a tra-

jectory, and the object of training is a policy that maximises the reward accumulated along trajectories it generates. In this work the policy is a neural network that maps the observation vector of Section 3.3.1 to a distribution over six joint-position targets, and training adjusts the network weights.

Algorithms of this kind divide into two families by what data they are allowed to learn from. An *on-policy* method updates the policy using only transitions collected by the policy as it currently stands; once an update changes the policy, the data that produced it are discarded. An *off-policy* method keeps a replay buffer and reuses transitions collected by earlier versions of the policy, sometimes many times over. The trade is between sample efficiency and stability. An off-policy method extracts far more learning from each interaction, which is what matters when interactions are expensive, as they are on physical hardware. An on-policy method throws data away but always optimises against the distribution its current policy actually visits, which makes its updates easier to reason about and its behaviour easier to reproduce.

This study is on-policy, and the reason is that its samples are cheap. At 4096 environments running in parallel on one GPU the bottleneck is the gradient computation, not the collection of experience, so the sample efficiency an off-policy method would buy has little to pay for itself with. Both families are established on robotic grasping [17], so this is a positioned choice rather than the only available one, and Chapter 6 returns to the off-policy comparison as the recognised complement to what is reported here.

3.1.3 Proximal policy optimisation and its constrained variant

Proximal Policy Optimisation, PPO, is the on-policy method used throughout this thesis [14]. Three of its mechanics are relevant. It optimises a clipped surrogate objective, which caps how far one update is permitted to move the policy away from the one that collected the data and removes the need to tune a step size against divergence. It estimates the advantage of an action by generalised advantage estimation, which trades bias against variance through a smoothing parameter. And it performs several epochs of minibatch updates on each batch of collected experience, recovering part of the sample efficiency an on-policy method gives up.

The constrained variant used here, written cPPO throughout, is PPO with three additions and no other change. It carries a second critic network that predicts accumulated future *cost* in the same way the ordinary critic predicts accumulated future reward. It carries a scalar Lagrange multiplier that weighs cost against reward inside the advantage. And it updates that multiplier once per iteration from the constraint violation the last batch actually exhibited, so the weight is set by measurement rather than by the designer. The method is PPO-Lagrangian as specified and benchmarked by Ray et al. [2], within the constrained policy optimisation family founded by Achiam et al. [11]. Section 3.6 gives the equations.

3.1.4 The UR5e manipulator

The platform is a Universal Robots UR5e, a six-degree-of-freedom collaborative arm. Its published specifications are given in Table 3.1 [9], with the modelling decision each one drives, because several of the numbers that appear later in this chapter are the arm's datasheet limits carried into the simulator rather than values chosen for training convenience.

Table 3.1. UR5e specifications and where each one enters the method.

Property	Value	Where it is used here
Degrees of freedom	6 revolute	Six-dimensional action space (Section 3.3.1)
Payload	5 kg	Not exercised; the grasp is a weld (Section 3.3)
Reach	850 mm	Bounds the goal-sampling box, far corner 0.84 m
Repeatability	$\pm$0.03 mm	Sets the scale at which a 1 cm goal tolerance is loose
Mass	20.6 kg	Not exercised; the arm is fixed-base in simulation
Maximum joint speed	180 deg/s (π rad/s)	Velocity limit in the simulated articulation
Safety functions	17, configurable	**Not used.** The learned constraint is what is under test
Certification	ISO 13849-1 Cat. 3 PL d; ISO 10218-1	Context only; no certified function is simulated

The last two rows carry a distinction that the rest of the chapter depends on. A real UR5e already has a safety controller, and it is a certified one. Nothing in this thesis replaces it, competes with it, or is evaluated against it. What the simulation borrows from the platform is its physical envelope, its reach and its speed ceiling. What the simulation tests is whether a learned policy can be held inside a kinematic safety budget by the optimiser itself, which is a different question from whether a hardware interlock can stop the arm.

Key points

- Isaac Sim solves the physics on the GPU and Isaac Lab declares the task through managers, which is what makes thousands of parallel environments and a hardware retarget practical.
- On-policy learning is chosen because simulated samples are cheap at 4096 environments, not because off-policy methods are unsuitable.

- cPPO is PPO plus a cost critic, a Lagrange multiplier, and a rule for updating that multiplier from measured violation. Nothing else differs.
- The arm's reach and joint-speed limits come from its datasheet and are carried into the simulator. Its seventeen built-in safety functions are deliberately not used, since they are not what is under test here.

3.2 Problem formulation

The grasping task is posed as a constrained Markov decision process, or CMDP [13]. A standard Markov decision process is the tuple (S, A, P, r, γ) over a state space S, an action space A, a transition kernel P giving the probability of the next state under a chosen action, a reward function r and a discount factor γ. A CMDP augments that tuple with a cost function c, which scores each transition for hazard exactly as r scores it for progress, and a budget d. It asks for the policy that maximises expected discounted return subject to the expected accumulated cost staying within budget:

$$\max_{\pi} \mathbb{E}\left[\sum_{t} \gamma^{t} r(s_t, a_t)\right] \quad \text{subject to} \quad \mathbb{E}\left[\sum_{t} c(s_t, a_t)\right] \leq d. \tag{3.1}$$

The formulation separates *what the robot should achieve*, which is to lift the cube and bring it to a commanded pose, from *what it must not do*, which is to pass through configurations where the arm loses controllability or approaches its mechanical limits. The first is encoded in r and the second in c, and the two never mix. This is the structural property the whole comparison in Chapter 4 rests on: because no safety term appears in the reward, any safety difference measured between a constrained agent and an unconstrained one is attributable to the constraint machinery and not to a reward the two agents were optimising differently.

Three properties of Equation 3.1 shape how the results must be read, and stating them here avoids over-claiming later.

The constraint binds an **expectation, not a maximum**. A policy satisfying it is one whose *average* accumulated cost over episodes falls within d, which permits individual episodes to exceed the budget provided others compensate. A hardware safety case therefore has to be argued in terms of expected exposure over many operations, not in terms of a guaranteed worst case [15], and Chapter 4 reports a counter-result where the single worst episode is no better under the constraint than without it.

The cost is **undiscounted and episodic**. Where the return in Equation 3.1 is discounted by γ, the constrained quantity is a plain sum over a fixed 250-step episode. Hazard late in an episode therefore counts as heavily as hazard early in it, which is the correct treatment for a physical limit but ties the numerical value of d to the episode length. Changing the episode

duration rescales the constraint and voids its calibration, a point Section 3.7 returns to.

The budget is **a single scalar over an aggregated cost**. Section 3.5 sums three separate hazard terms into one number before the constraint sees it, so one multiplier governs all three and the agent is free to trade between them. This keeps the optimisation simple at the price of not being able to state a separate guarantee per hazard, and the three terms are therefore reported separately even though they are constrained jointly.

Solving Equation 3.1 directly is awkward, since the constraint couples every timestep of every episode. The standard treatment, and the one used here, converts it into an unconstrained saddle-point problem through a Lagrange multiplier, which Section 3.6 develops. The remainder of the chapter fills in each element of the tuple in turn: the simulation environment and the MDP itself (Section 3.3), the software that implements them (Section 3.4), the safety cost that instantiates c (Section 3.5), the algorithm that solves the CMDP (Section 3.6), the calibration that makes the constraint meaningful rather than decorative (Section 3.7), and the training and evaluation protocol (Sections 3.8–3.9).

Key points

- The task is a CMDP: maximise expected return subject to expected accumulated cost staying within a budget d.
- Reward and cost are kept in separate channels, which is what makes the Chapter 4 comparison attributable to the constraint.
- The constraint binds an average, not a worst case, so the safety claim is about expected exposure rather than a guarantee.
- The cost is undiscounted over a fixed-length episode, so d is only meaningful at the episode length it was calibrated against.

3.3 Simulation environment and task

Experiments are carried out in NVIDIA Isaac Sim 5.0.0 with the Isaac Lab 2.3.0 manager-based framework [20], using the `rsl_rl` 3.0.1 reinforcement-learning library [22], PyTorch 2.7 (CUDA 12.8) and a single RTX 5090 GPU. Physics, observation construction and policy inference all run on the GPU without copying state to the host, which is what makes training at several thousand simultaneous environments practical: the same design applied to earlier manipulation and locomotion tasks gave speed-ups of two to three orders of magnitude over CPU simulators [21], and it is the reason the 4096-environment setting used in Section 3.8 is a reasonable default rather than an unusually large one. The task is retargeted from Isaac Lab's Franka cube-lift environment onto a Universal Robots UR5e equipped with a Robotiq 2F-85 gripper; only the robot, action space, and end-effector frame are changed, so the

reward shaping and task structure of the well-tested base environment are preserved.

The manipulator is a six-degree-of-freedom arm controlled by joint-position commands on its six revolute joints (shoulder pan, shoulder lift, elbow, and three wrist joints), with the action scaled by 0.5 about the default configuration. The gripper is driven by a single binary open/close command on the Robotiq actuation joint. The object is a rigid cube (Isaac's DexCube asset scaled to 0.8) initialised on a table in front of the arm, and its target is a goal pose sampled once per episode.

Because the Robotiq 2F-85 is a closed-loop linkage whose passive finger joints transmit no normal force to the pads in simulation, contact-based grasping does not hold the cube (verified with `grasp_hold_test.py`: the cube falls through the fingers regardless of clamp force). A subsequent diagnostic (`tools/check_gripper_mount.py`) identified the underlying mechanism: in the converted USD asset, all nine gripper rigid bodies are reported by the physics articulation view at positions coincident with the `wrist_3_link` origin, rather than distributed along the 150 mm gripper body. The pad-to-pad separation is nevertheless reproduced correctly (84.4 mm measured against an 85 mm specification stroke), confirming that the gripper's internal linkage is intact while its transform relative to the wrist is degenerate. Finger contact therefore cannot resolve at the location where the gripper geometry is rendered, which fully accounts for the observed pass-through behaviour.

Since the Layer 1 safe-RL result is independent of the specific grasp mechanism, the gripper is modelled as a lumped mass at the wrist flange and contact grasping is replaced by a *proximity-weld* abstraction. The tool centre point is defined kinematically as a fixed 160 mm offset along the `wrist_3_link` approach axis (the position a correctly mounted 2F-85 pad midpoint would occupy), and when the policy commands a close action while the cube lies within a 6 cm tolerance of that frame, the cube latches and its pose tracks the end-effector each control step (with its velocity zeroed); an open command releases it and physics resumes. This makes "grasp-and-lift" reliable so that both the baseline and the constrained policy can learn the task, and defers realistic finger contact to the Layer 3 sim-to-real work.

Two consequences of this abstraction are stated explicitly. First, the lumped-mass gripper alters the wrist inertia relative to a fully articulated model; because the same asset is used for every run in the benchmark, this affects all algorithms identically and does not confound the comparison. Second, self-collision is disabled in simulation and the collision cost term is a table-plane keep-out rather than a link-pair check, so learned configurations are not guaranteed to be self-collision-free on hardware. Both are revisited in the Limitations discussion.

3.3.1 State, action, and reward

The policy receives a privileged observation comprising the arm joint positions and velocities (relative to their defaults), the cube position expressed in the robot base frame, the com-

manded goal position, and the previous action. Object pose is provided directly rather than through vision, which isolates the safe-RL contribution from perception; closing the loop with an image-based estimate is the subject of Layer 2. All observation terms are clamped to a finite range as a numerical safeguard, preventing a single transiently-unstable environment from corrupting the on-policy batch.

The action is the six-dimensional vector of target arm-joint positions together with the binary gripper command. Episodes last five simulated seconds, or 250 control steps at 50 Hz, with a control decimation of two physics steps per action. The goal pose is resampled once per episode from a uniform range over the workspace, $x \in [0.22, 0.60]$ m, $y \in [-0.30, 0.30]$ m, $z \in [0.10, 0.50]$ m. That box is bounded by reachability rather than by convenience: with the UR5e base at the environment origin and a rated reach of about 0.85 m, its far corner sits 0.84 m from the base, leaving roughly 13 mm of margin, so no sampled goal is physically unreachable. At reset the cube position is randomised within a small planar region so the policy cannot memorise a fixed trajectory.

The reward is the shaped sum inherited from the base lift task, with weights retuned for this goal box. A reaching term rewards reducing the end-effector-to-object distance (tanh kernel, standard deviation 0.1, weight 1). A lifting term rewards raising the cube (weight 10). Two goal-tracking terms reward bringing the lifted cube toward the commanded goal at coarse and fine scales (tanh kernels with standard deviations 0.3 and 0.05, weights 15 and 5). Small quadratic penalties on the action rate and joint velocity (each weight -10^{-4}) discourage jerky motion. All three lift-dependent terms gate on the cube having climbed half of the distance from the table to the goal height sampled for that episode, rather than on a fixed absolute height, because a fixed threshold scales badly once the goal box spans 0.10 m to 0.50 m in z. Shaped distance-and-lift rewards of this form, tuned per task rather than derived, are the standard construction in learned contact-rich manipulation [16], so the design is conventional and only its safety treatment is not.

The reward contains **no safety term**. Safety is expressed entirely through the separate cost channel of Section 3.5, so that the comparison in the Results chapter isolates the effect of the constraint.

Table 3.2 collects the environment as a whole.

Table 3.2. Summary of the simulation environment.

Item	Setting
Simulator	Isaac Sim 5.0.0, PhysX, GPU-resident
Framework	Isaac Lab 2.3.0, manager-based
Task	`Isaac-Lift-Cube-UR5e-v0`, retargeted from the Franka cube-lift task
Robot	UR5e, 6 revolute joints, fixed base at the environment origin
End effector	Robotiq 2F-85, modelled as a lumped mass at the wrist flange
Grasp model	Proximity weld, 160 mm TCP offset, 6 cm latch tolerance
Action space	6 joint-position targets (scale 0.5) + 1 binary gripper command
Observation	Joint positions and velocities, cube position, goal position, previous action
Episode length	5.0 s = 250 control steps at 50 Hz, decimation 2
Goal box	x [0.22, 0.60], y $[-0.30, 0.30]$, z [0.10, 0.50] m; far corner 0.84 m
Reward terms	Reaching (1), lifting (10), goal tracking coarse (15) and fine (5), action-rate and joint-velocity penalties (-10^{-4} each)
Lift gate	50 % of the climb from the table to that episode's goal height
Safety terms in reward	None. Safety is a separate cost channel
Self-collision	Disabled; collision cost is a table-plane keep-out
Parallel environments	4096 (training), 128 (evaluation)

Key points

- The task is Isaac Lab's Franka cube-lift environment retargeted onto a UR5e, so the reward shaping and termination logic come from a tested base rather than being written from scratch.
- Contact grasping does not hold the cube because the converted gripper asset reports all

nine of its bodies at the wrist origin, diagnosed with `check_gripper_mount.py`. The grasp is therefore a proximity weld, and Layer 3 is where real finger contact belongs.
- The weld and the lumped-mass gripper are applied identically to every arm in the benchmark, so they cannot confound the comparison, but they do mean the success rates here are not comparable to a full physical grasp.
- The goal-sampling box is bounded by the arm's 850 mm reach, not chosen for convenience.

3.4 Software framework

Every piece of code written for this thesis lives in one Python package, `ur5_grasp`, installed beside Isaac Lab rather than inside it. Keeping it outside the simulator's own source tree has two practical consequences. The environment, the safety cost and the constrained algorithm can be committed, frozen and tagged as a single unit, which is what makes a run reproducible against a named commit. It also keeps the boundary visible between what this work contributes and what it inherits from the framework. Table 3.3 lists the modules.

The safety cost is computed by `SafetyCostComputer`. It resolves the arm joint indices, the monitored body indices and the Jacobian body-axis index once on the first call and caches them, and all arithmetic is batched across environments with nothing copied back to the host. At 4096 environments a per-step host synchronisation would cost more than the physics step it is measuring. The class returns the aggregate cost that the constraint acts on, and alongside it a mean value and a binary violation indicator for each of the three terms, so the constraints can be reported separately even though a single multiplier governs them jointly.

The constrained agent extends `rsl_rl` [22] across four modules rather than replacing it with a different library. `ActorCriticCost` adds a cost critic beside the actor and the reward critic, and builds it after the base class has finished, so the actor and reward critic draw their initial weights from the same random state as the unconstrained baseline's. `RolloutStorageCost` carries the cost stream through the rollout buffer and computes cost advantages with their own discount and smoothing. `PPOLagrangian` implements the combined advantage and the dual update given in Section 3.6. `LagrangianRunner` replaces a single method of the stock runner, `_construct_algorithm`, which is necessary because the stock runner resolves class names by evaluating them inside its own module namespace, where classes defined outside the library are not visible. The rollout loop, the logging and the checkpointing are inherited unchanged, so the multiplier, the mean episodic cost and the two critic losses reach TensorBoard without additional code.

Training and evaluation each run from one script. `scripts/train.py` launches every arm, and the arm is selected by an agent-configuration entry point, so the baseline and the constrained agents are started by the same command with one flag changed and can differ in

Table 3.3. The ur5_grasp package.

Module	Role
robots/ur5e_robotiq.py	UR5e articulation, actuator gains, joint speed and effort limits
tasks/lift/ur5e_lift_env_cfg.py	Task configuration: action space, goal-sampling box, reward terms
tasks/lift/ur5e_lift_env.py	Environment class, safety thresholds, per-step cost and safety/* logging
tasks/lift/rewards.py	Goal-relative lift gate used by the three lift-dependent reward terms
safe_rl/costs.py	SafetyCostComputer: the three cost terms of Section 3.5
safe_rl/actor_critic_cost.py	Actor, reward critic and cost critic
safe_rl/rollout_storage_cost.py	Rollout buffer carrying the cost stream and cost advantages
safe_rl/ppo_lagrangian.py	Combined advantage, dual update, partitioned gradient clip
safe_rl/lagrangian_runner.py	Runner that assembles the constrained agent
tasks/lift/agents/	Per-arm hyperparameter configurations (baseline, control, cPPO)
scripts/train.py	Training entry point for every arm
scripts/eval_policy.py	Evaluation of a frozen checkpoint (Section 3.9)
tools/	Asset builders, gripper diagnostics, run summarisers, regression tests

nothing else. `scripts/eval_policy.py` loads a saved checkpoint and scores it under the protocol of Section 3.9. Shell launchers wrap both for batch execution across arms and seeds, and each run writes its resolved environment and agent configuration to disk beside its checkpoints, so two runs can be compared by differencing their dumped configurations instead of by trusting that the same flags were passed.

3.4.1 The gradient-clip audit

An earlier version of this benchmark, trained on 2026-07-30, reported a large advantage for the constrained agent on both reward and safety. That result was withdrawn. The reasons are recorded here because the correction shaped the software described above and the experimental design that follows, and because Section 4.2 reports the verification that the correction worked.

The first sign was in the logged multiplier. `Loss/cost_lambda` read 0.0 at almost every iteration of all three constrained runs, and on one seed at every iteration without exception. Setting $\lambda = 0$ in Equation 3.3 leaves $A = A_{\text{reward}}$, so the update in those runs was algebraically stock PPO and the constraint cannot have been what separated the arms. Something else had.

The mechanism was in the gradient clip. Stock PPO in `rsl_rl` [22] applies one gradient-norm clip across every parameter handed to the optimiser, computing a single global norm and rescaling all gradients by the same factor. In the constrained agent that parameter list also holds the cost critic, so the cost critic's gradients entered the norm and scaled down the actor's step on every update, including all the updates where the multiplier was exactly zero. With the maximum gradient norm set to 1.0 the clip binds often. The constrained agent was therefore not PPO plus a constraint; it was PPO with a smaller and cost-dependent effective step size, and a quieter optimiser is a well-known stabiliser on shaped-reward manipulation tasks.

Two further faults surfaced in the same audit. The estimate of mean episodic cost that drives the dual update was accumulated in a 100-slot buffer, while 4096 environments reach their fixed horizon on the same step, so the multiplier was reacting to about 2.4 per cent of each batch. And the budget of 25 sat above the natural episodic cost of the converged runs, so the dual update was correct to hold the multiplier at zero. A constraint that is never exceeded measures nothing.

The fix partitions the parameters into two groups, the actor together with the reward critic in one and the cost critic in the other, and clips each group independently, so the baseline half of the network receives treatment identical to the unconstrained agent's. The cost buffer was resized to hold a full wave of simultaneously terminating episodes, and the number of episodes actually entering each estimate is now logged rather than assumed. A regression test, `tools/test_grad_clip_fix.py`, checks the three claims the correction rests on in

about two seconds of pure PyTorch without launching the simulator: that one global clip does change the actor's gradients relative to a two-group clip, that the two-group clip restores gradients identical to the baseline's, and that the combined advantage reduces exactly to the reward advantage when the multiplier is zero.

A fix of this kind cannot be verified by reading the code, so a third arm was added to the experiment. The control arm carries the cost critic and everything else the constrained agent carries, including the offset it causes in the random-number stream, but pins the multiplier ceiling to zero so the constraint can never engage. Comparing the control arm against an unconstrained baseline isolates whatever remains of the implementation artifact. Comparing the constrained agent against the control arm isolates the constraint. Only the second comparison is a safe reinforcement learning result, and Section 4.2 reports both.

Key points

- All contributed code sits in one package outside the simulator tree, so a run is reproducible against a single tagged commit.
- The constrained agent is a thin extension of the same trainer the baseline uses, which is what lets a measured difference be attributed to the constraint rather than to two different implementations of PPO.
- An earlier version of this benchmark was withdrawn. One global gradient clip spanning the cost critic had been quietly shrinking the actor's step, so the constrained agent was PPO with a quieter optimiser rather than PPO with a constraint.
- The fix is a two-group clip, verified by a regression test and, more importantly, by a control arm that carries the cost critic with the constraint switched off.

3.5 The safety cost function

Safety is encoded by a per-step cost, computed by the cost class of Section 3.4 from three geometric-kinematic terms. Each term is zero while the arm is safe and grows smoothly as the arm enters a danger zone; smoothness matters because it gives the cost critic (Section 3.6) a usable gradient. The three terms are summed into a single aggregate cost so that a single Lagrange multiplier governs the whole constraint, and each term additionally exposes a mean value and a binary violation indicator for reporting.

The first term is a **collision keep-out**. For each monitored arm link (forearm, wrist-1, wrist-3) the encroachment below a floor height, $\max(z\_floor - z, 0)$, is summed across links, with $z\_floor = 0.05$ m. The floor is a 5 cm standoff above the table rather than the table surface itself, so the term begins to cost before a link actually penetrates anything. Because self-collisions are disabled in simulation and no contact sensors are present, this geometric proxy

is the honest, inexpensive choice.

The second term is a **joint-limit margin**. For each arm joint the clearance to its nearer soft limit is computed, and an encroachment penalty clamp(1 − clearance / margin, 0, 1) is summed over the six joints, with a margin of 0.175 rad (about 10.0°). The penalty is zero until a joint enters the final margin band and rises linearly to one at the limit.

The third term, and the operative constraint of this thesis, is a **singularity floor**. The Yoshikawa manipulability measure w = $\sqrt{\det(JJ^\top)}$ [18] is computed from the 6×6 end-effector Jacobian J of the arm, where a small w indicates a near-singular configuration in which the arm momentarily loses the ability to move in some Cartesian direction and commands can produce large, jerky joint velocities. The penalty clamp(1 − w / w_floor, 0, 1) activates as w falls below a floor of 0.06 and reaches one as w approaches zero. The Jacobian body-axis index is resolved automatically to accommodate the fixed-base articulation (for which PhysX omits the root body from the Jacobian), and correctness was confirmed at calibration by a smooth, small-positive distribution of w.

The aggregate per-step cost is c = c_collision + c_joint + c_singularity (unit weights), and its undiscounted sum over an episode is the quantity constrained against the budget d.

Key points

- Three hazards are scored: proximity to the table, proximity to a joint limit, and proximity to a kinematic singularity.
- Every term is smooth rather than a switch, because the cost critic needs a gradient to learn from. A binary violation flag would give it none.
- The singularity term uses the Yoshikawa manipulability measure and is the constraint this thesis is built around; the joint-limit term turned out to be active as well, which Section 3.7 explains.
- The three are summed into one number before the constraint sees it, so one multiplier governs all three, but each is logged separately for reporting.

3.6 Constrained policy optimisation (cPPO)

The unconstrained baseline of this study is Proximal Policy Optimisation [14]: a clipped surrogate objective, generalised advantage estimation, and several epochs of minibatch updates over each on-policy rollout. The CMDP is solved with the Lagrangian variant of that algorithm, hereafter cPPO, which is the PPO-Lagrangian method specified and benchmarked by Ray et al. [2] within the constrained policy optimisation family founded by Achiam et al. [11]. Their benchmark is also the reason the Lagrangian variant is preferred here over CPO itself, which it reports as the weaker of the two on the same tasks. The method converts the

constrained problem into the saddle-point objective

$$\max_{\pi} \min_{\lambda \geq 0} \quad J_{\text{reward}}(\pi) - \lambda \left(J_{\text{cost}}(\pi) - d \right), \tag{3.2}$$

where λ is a non-negative Lagrange multiplier acting as an automatically-tuned penalty on constraint violation. The implementation, whose modules are listed in Table 3.3, follows the textbook separate-critic variant: in addition to the usual value network that estimates future reward, a second critic estimates future cost, so reward and cost advantages are learned independently. Cost advantages are computed by generalised advantage estimation with its own discount and smoothing (γ_cost = 0.98, λ_GAE = 0.95).

At each policy-update step the reward and cost advantages are combined into a single Lagrangian advantage,

$$A = \frac{A_{\text{reward}} - \lambda A_{\text{cost}}}{1 + \lambda}, \tag{3.3}$$

which is then used in the standard clipped PPO surrogate; the division by (1 + λ) keeps the effective step size stable as λ grows. The multiplier itself is updated once per iteration by projected dual ascent on the measured mean episodic cost of the just-collected rollout,

$$\lambda \leftarrow \text{clip}\big(\lambda + \eta \, (\hat{J}_{\text{cost}} - d), \; 0, \; \lambda_{\max}\big), \tag{3.4}$$

with dual step η = 0.035, initial λ = 0, and ceiling λ_max = 100. Intuitively, whenever the policy exceeds its cost budget the multiplier rises and unsafe actions are penalised more heavily; once the policy is comfortably under budget the multiplier decays back toward zero. The multiplier tunes itself during training and is never hand-set.

The unconstrained baseline is exactly this algorithm with λ pinned at zero, which is stock PPO on the same `rsl_rl` trainer with no cost critic. Building the constrained agent as a thin extension of the baseline, rather than adopting a different safe-RL library, is a deliberate methodological choice: it guarantees that the baseline and the constrained policy share an identical trainer, network architecture, and hyperparameters, so any measured difference is attributable to the safety constraint alone and not to two dissimilar PPO implementations.

Both agents use the same actor and critic multilayer perceptrons with hidden layers of 256, 128 and 64 units and ELU activations, and the same optimisation hyperparameters, summarised in Table 3.4.

Table 3.4. Shared training hyperparameters.

Hyperparameter	Value
Parallel environments	4096
Rollout length (steps/env)	24
Iterations	1500
Actor / critic hidden layers	[256, 128, 64], ELU
Clip parameter	0.2
Entropy coefficient	0.006
Learning epochs / mini-batches	5 / 4
Learning rate (adaptive, target KL 0.01)	1×10^{-4}
Discount γ / GAE λ	0.98 / 0.95
Max gradient norm (per group)	1.0
cPPO cost budget d (`cost_limit`)	25, and 15 for the tighter arm
cPPO dual step η / λ ceiling	0.035 / 100
cPPO cost discount / GAE λ	0.98 / 0.95

Key points

- The constrained problem becomes a saddle point: the policy maximises reward while the multiplier pushes back in proportion to how far the cost budget is exceeded.
- The multiplier is set by dual ascent on measured violation, so the safety-versus-task weight is never hand-tuned. This is the property a shaped reward cannot offer.
- Reward and cost advantages are estimated by separate critics, then combined and divided by $(1 + \lambda)$ so the effective step size does not grow with the multiplier.
- The baseline is the same algorithm with the multiplier pinned at zero, sharing trainer, architecture and hyperparameters, which is what makes the arms comparable.

3.7 Constraint calibration and cost budget

A safety benchmark is only meaningful if the unconstrained baseline actually violates the constraint; otherwise the constrained policy has nothing to demonstrate. The three thresholds and the cost budget were therefore calibrated from the real trained baseline rather than guessed.

All three thresholds and the budget were set from a converged 1500-iteration unconstrained agent run against the final environment, immediately before the code was frozen and tagged.

Calibrating against a converged policy rather than a short probe matters here, because a policy fifty iterations into training has barely begun to reach and its cost distribution says little about the one that would be deployed.

The manipulability floor was set from that agent's distribution of w, which had a minimum of 0.0171, a mean of 0.0750 and a maximum of 0.1109, with the tenth and twenty-fifth percentiles at 0.0517 and 0.0682. A floor of 0.06 sits at about the eighteenth percentile and makes the unconstrained baseline violate the constraint on roughly a fifth of its steps, which is the design intent: a floor the baseline never reaches would leave the constrained agent nothing to demonstrate. The floor was raised to 0.06 from an earlier value of 0.045, which had been set against a narrower goal-sampling box and no longer produced a violation rate inside that band once the workspace was widened.

The joint-limit margin of 0.175 rad is a second active constraint. Under this goal-sampling box the baseline spends 33.7 per cent of its steps inside that margin and its minimum clearance reaches the soft limit itself, so joint-limit proximity is not the monitored-but-satisfied term it was under the narrower workspace of earlier drafts. It is in fact the larger of the two active terms, accounting for roughly 86 per cent of the natural episodic cost against the singularity term's 14 per cent. The collision floor of 0.05 m stays inactive, with a minimum monitored-link height of 0.0896 m, though the clearance above the floor is now about 44 mm where it was 125 mm before the workspace widened. Two active constraints share one budget and one multiplier, so the constrained agent is free to trade between them, and Chapter 4 reports each separately for that reason.

The episodic cost budget was set against the same converged agent, whose natural episodic cost is about 105. Holding d = 25 asks for roughly a 76 per cent reduction. The budget is an undiscounted sum over a per-step cost across a fixed 250-step episode, so it is tied to the five-second episode length: changing the episode length would rescale the constraint and void this calibration.

Key points

- Thresholds were measured off a converged unconstrained agent, not guessed and not taken from a short probe, because a barely-trained policy says little about the one that would be deployed.
- The manipulability floor is placed so the unconstrained baseline violates it about a fifth of the time. A floor it never reaches would make the benchmark vacuous.
- Two constraints are active, not one. Joint-limit proximity accounts for roughly 86 % of the natural episodic cost and singularity for 14 %, which changes the framing carried into Chapter 4.

- The budget of 25 against a natural cost of about 105 asks for a 76 % reduction, and is valid only at the five-second episode length it was calibrated against.

3.8 Training protocol

Each agent was trained for 1500 iterations at 4096 parallel environments (roughly 2×10^5 environment steps per second on the RTX 5090), headless, with progress monitored through TensorBoard. The constrained and baseline agents are launched from the same script and differ only by an agent-config flag, and they log to separate experiment directories so their curves overlay directly for comparison. The healthy Lagrangian signature to be verified during training is that the mean episodic cost trends down toward the budget, the multiplier rises while over budget and then settles without pinning at its ceiling, the reward continues to improve, and the singularity-violation rate falls below the baseline.

3.8.1 Experimental arms and seeds

Three arms were trained. The baseline is the control arm described in Section 3.4.1, which carries the cost critic with its multiplier ceiling pinned to zero and is reported throughout as PPO (baseline).[1] The two constrained arms are cPPO at a budget of 25 and cPPO at a tighter budget of 15, identical to each other in every other respect. The second budget is a sensitivity check on the calibration rather than a separate method: 15 binds more deeply than 25 on the seeds where either binds at all, so the pair shows whether the effect of the constraint scales with how hard it is applied.

Each arm was trained on five random seeds, 1, 3, 4, 52 and 54, giving fifteen trained policies. Reporting five seeds and their dispersion, rather than a single run or a mean alone, follows the reproducibility argument made by Henderson et al. [4], who show that deep policy-gradient results on continuous control can differ qualitatively between seeds of the same algorithm and that a mean over too few runs can describe a policy none of them produced. Dispersion across seeds is therefore reported as a result in its own right in Chapter 4, not as an error bar attached to one.

Table 3.5 states the protocol as run.

[1] A plain PPO arm was trained on the same seeds. Its stored network weights proved byte-identical to the control arm's, so the control arm stands in for it and the plain arm is not reported separately. The verification is given in Section 4.2 and the raw comparison in `Comparison_test/ppo_redundant/README.md`.

Table 3.5. Training protocol.

Item	Setting
Arms	3: PPO (baseline), cPPO at d = 25, cPPO at d = 15
Seeds per arm	5: 1, 3, 4, 52, 54
Trained policies	15
Iterations	1500 per run
Parallel environments	4096
Rollout length	24 steps per environment per iteration
Throughput	$\approx 2 \times 10^5$ environment steps/s on an RTX 5090
Library	`rsl_rl` 3.0.1, PyTorch 2.7, CUDA 12.8
Execution	Headless, monitored through TensorBoard
Launch	One script, arm selected by agent-config entry point
Logging	Separate experiment directory per arm, so curves overlay directly
Frozen at	Commit `567e4c0`, tag `matrix-v2`
Verification	All 15 final checkpoints confirmed present on disk, not inferred from clean logs

Key points

- Fifteen policies: three arms on five seeds each, all against one frozen commit.
- The arms are launched by the same script with one flag changed, so they cannot differ by anything unrecorded.
- Dispersion across seeds is treated as a finding, not as noise to be averaged away.
- The healthy signature to watch during training is the episodic cost falling toward the budget while the multiplier rises, settles, and does not pin at its ceiling.

3.9 Evaluation protocol

Every reported number is measured after training on the frozen policy, not read from training-time telemetry. This distinction is a correction rather than a preference. An earlier version of this work reported safety percentages taken from the TensorBoard scalars of a policy that was still exploring and still learning, which describes neither the policy that would be deployed

nor any policy at all. Each checkpoint is therefore replayed by `eval_policy.py` with the actor set to its deterministic mean action, exploration noise off and observation corruption disabled.

Each of the fifteen trained policies is evaluated over 1000 completed episodes at each of three evaluation seeds, 101, 102 and 103, none of which appear among the training seeds, using 128 parallel environments. Every arm is therefore scored over 15,000 episodes and every individual policy over 3000. Using evaluation seeds disjoint from the training seeds separates variation caused by the evaluation itself from variation caused by which policy was learned, and the two turn out to differ by more than an order of magnitude.

A *lift success* is recorded when the cube is raised past half of the climb from the table to that episode's commanded goal height, matching the reward's own gate. A *goal-reach success* is recorded when the lifted cube is additionally brought within 1 cm of the commanded goal pose, with the goal position transformed into the world frame exactly as in the reward computation. The goal-reach figure is reported alongside the full distribution of final goal distances rather than on its own, because the weld abstraction writes the cube's pose to the tool frame at every step, so the 1 cm test reduces to whether the tool centre point reached the goal. A converged policy passes that test on almost every episode and an unconverged one fails it on almost every episode, which makes a single threshold saturate and hide the difference between policies that a distribution shows.

Safety is counted per episode rather than averaged per step. The reported quantities are the episodic safety cost, which is the exact quantity the Lagrangian constrains, the fraction of episodes containing a true singularity crossing (w below 10^{-4}), and the fraction of episodes in which any joint touched its limit margin at all. For the constrained arms the terminal multiplier and the episodic-cost trajectory are reported as well, since they document whether the constraint engaged. The resulting comparison is presented in Chapter 4. Table 3.6 states the protocol.

Table 3.6. Evaluation protocol.

Item	Setting
Policy	Frozen checkpoint, deterministic mean action, exploration noise off
Observation corruption	Disabled
Evaluation seeds	101, 102, 103, disjoint from the training seeds
Episodes	1000 per checkpoint per evaluation seed
Parallel environments	128
Episodes per policy	3000
Episodes per arm	15,000
Lift success	Cube past 50 % of the climb to that episode's goal height
Goal-reach success	Lifted cube within 1 cm of the commanded goal pose
Reported alongside	Full distribution of final goal distances, since the weld makes a single threshold saturate
Safety, per episode	Episodic cost; singularity crossings ($w < 10^{-4}$); episodes touching the joint-limit margin
Constrained arms only	Terminal multiplier, episodic-cost trajectory
Source of all figures	`Comparison_test/final_results/`

Key points

- Everything reported is measured on the frozen deterministic policy. Reading safety off training-time telemetry was a fault in an earlier version of this work, not a stylistic choice.
- Evaluation seeds are disjoint from training seeds, which separates variation caused by the exam from variation caused by the policy being examined.
- Goal-reach is reported with its distance distribution, because the weld collapses the 1 cm test into a reaching problem that a converged policy passes almost always and an unconverged one almost never.
- Safety is counted per episode rather than averaged per step, so a single dangerous excursion is visible instead of being diluted across 250 steps.

BIBLIOGRAPHY

[1] Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16(42):1437–1480, 2015.

[2] Alex Ray, Joshua Achiam, and Dario Amodei. Benchmarking safe exploration in deep reinforcement learning. Technical report, OpenAI, 2019.

[3] Yue Shen, Qingxuan Jia, Zeyuan Huang, Ruiquan Wang, Junting Fei, and Gang Chen. Reinforcement learning-based reactive obstacle avoidance method for redundant manipulators. *Entropy*, 24(2):279, 2022.

[4] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1):3207–3214, 2018.

[5] Fawad Khan, Wei Feng, Tianlun Huang, Zhiyong Wang, Xiao Liu, Shahid Asad Ali, and Weijun Wang. Reinforcement learning for precision grasping and safety-critical coordination in a robotic arm. *Intelligent Service Robotics*, 19(16), 2026.

[6] Weidong Xia, Yuqian Lu, Wenjun Xu, and Xun Xu. Deep reinforcement learning based proactive dynamic obstacle avoidance for safe human-robot collaboration. *Manufacturing Letters*, 41:1246–1256, 2024.

[7] Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by PID Lagrangian methods. In *Proc. 37th Int. Conf. Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 9133–9143. PMLR, 2020.

[8] Henrique C. Ferreira and Ramiro S. Barbosa. Deep reinforcement learning for adaptive robotic grasping and post-grasp manipulation in simulated dynamic environments. *Future Internet*, 17(10):437, 2025.

[9] Universal Robots A/S. UR5e technical specifications. Product datasheet, August 2023.

[10] Md Masrul Khan. *Enhancing Manipulator Control Through Image-Based Visual Servoing Techniques Using ROS2*. BSc thesis, Dept. of Mechatronics Engineering, Khulna University of Engineering & Technology, Khulna, Bangladesh, December 2025.

[11] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *Proc. 34th Int. Conf. Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 22–31. PMLR, 2017.

[12] Shangding Gu, Long Yang, Yali Du, Guang Chen, Florian Walter, Jun Wang, and Alois Knoll. A review of safe reinforcement learning: Methods, theories, and applications. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):11216–11235, 2024.

[13] Eitan Altman. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, Boca Raton, FL, USA, 1999.

[14] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[15] Lukas Brunke, Melissa Greeff, Adam W. Hall, Zhaocong Yuan, Siqi Zhou, Jacopo Panerati, and Angela P. Schoellig. Safe learning in robotics: From learning-based control to safe reinforcement learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 5:411–444, 2022.

[16] Íñigo Elguea-Aguinaco, Antonio Serrano-Muñoz, Dimitrios Chrysostomou, Ibai Inziarte-Hidalgo, Simon Bøgh, and Nestor Arana-Arexolaleiba. A review on reinforcement learning for contact-rich robotic manipulation tasks. *Robotics and Computer-Integrated Manufacturing*, 81:102517, 2023.

[17] Asad Ali Shahid, Dario Piga, Francesco Braghin, and Loris Roveda. Continuous control actions learning and adaptation for robotic manipulation through reinforcement learning. *Autonomous Robots*, 46:483–498, 2022.

[18] Tsuneo Yoshikawa. Manipulability of robotic mechanisms. *The International Journal of Robotics Research*, 4(2):3–9, 1985.

[19] Jiaming Ji, Borong Zhang, Jiayi Zhou, Xuehai Pan, Weidong Huang, Ruiyang Sun, Yiran Geng, Yifan Zhong, Juntao Dai, and Yaodong Yang. Safety gymnasium: A unified safe reinforcement learning benchmark. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023.

[20] Mayank Mittal, Calvin Yu, Qinxi Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlekar, Buck Babich, Gavriel State, Marco Hutter, and Animesh Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023.

[21] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac gym: High performance GPU-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021.

[22] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proc. 5th Conf. Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 91–100. PMLR, 2022.